

# Lecture 09 - Texturing

This lecture treats textures as sampled data for shading, not merely images glued onto objects. It connects UV coordinates, sampling, filtering, mipmaps, OpenGL texture state, and advanced texture uses.

|                       |                                                     |
|-----------------------|-----------------------------------------------------|
| Source chunk          | course_text_parts/03_lectures/09_texturing.txt      |
| Extracted slide pages | 133                                                 |
| Reader purpose        | Commentary and causal explanation for human reading |

## How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

## Per-slide commentary

Each extracted slide page gets its own note. Use this section as the spoken commentary that the terse slide deck is missing.

|                |                                                                                                                                                                                                                                                                                                                               |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 1         | Untitled slide                                                                                                                                                                                                                                                                                                                |
| Source cue     | The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image.                                                                                                                                                                                                                    |
| Commentary     | This slide is about Untitled slide. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image. |
| Why it matters | Even a sparse slide usually names a relation you must be able to explain in words.                                                                                                                                                                                                                                            |
| Check yourself | Can you turn 'Untitled slide' into a causal sentence instead of repeating the slide title?                                                                                                                                                                                                                                    |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 2         | Textures: Sampled Data for Shading                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Source cue     | ■ Textures are sampled data fields used during rendering / ■ Most common case: 2D image data mapped onto 3D geometry / ■ Texture values can represent color, transparency, normals, roughness, height, masks, or arbitrary / lookup data                                                                                                                                                                                                                        |
| Commentary     | This slide is about Textures: Sampled Data for Shading. Read it as local shading. Identify normal, light direction, view direction, material coefficients, and where the computation is evaluated. The visible cue is: ■ Textures are sampled data fields used during rendering / ■ Most common case: 2D image data mapped onto 3D geometry / ■ Texture values can represent color, transparency, normals, roughness, height, masks, or arbitrary / lookup data |
| Why it matters | Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.                                                                                                                                                                                                                                                                                                                                                          |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Check yourself | Can you identify the normal, light vector, view vector, and material term relevant to 'Textures: Sampled Data for Shading'?                                                                                                                                                                                                                                                                                                                                                         |
| Page 3         | Texturing Example                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| Source cue     | The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image.                                                                                                                                                                                                                                                                                                                                                                          |
| Commentary     | This slide is about Texturing Example. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image.                                                                                                                                                    |
| Why it matters | Even a sparse slide usually names a relation you must be able to explain in words.                                                                                                                                                                                                                                                                                                                                                                                                  |
| Check yourself | Can you turn 'Texturing Example' into a causal sentence instead of repeating the slide title?                                                                                                                                                                                                                                                                                                                                                                                       |
| Page 4         | Texture Mapping                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Source cue     | ■ Texture mapping / • Application of texture defined in multidimensional texture space to object defined in 3D space / • Realized through per-fragment operations / • A texture lookup turns texture coordinates into data used by the shader                                                                                                                                                                                                                                       |
| Commentary     | This slide is about Texture Mapping. Read it as concrete API state and GPU data movement. Ask which object is bound, which shader stage consumes it, and which state affects the draw call. The visible cue is: ■ Texture mapping / • Application of texture defined in multidimensional texture space to object defined in 3D space / • Realized through per-fragment operations / • A texture lookup turns texture coordinates into data used by the shader                       |
| Why it matters | OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.                                                                                                                                                                                                                                                                                                                                                                             |
| Check yourself | Can you name the OpenGL object, state, shader stage, or buffer involved in 'Texture Mapping'?                                                                                                                                                                                                                                                                                                                                                                                       |
| Page 5         | Why Textures Matter                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Source cue     | ■ Geometry defines shape / ■ Textures define appearance detail / ■ This separates visual complexity from geometric complexity / ■ Instead of modeling every scratch, brick, pore, label, or color variation, we store it in texture data / ■ Textures are efficient                                                                                                                                                                                                                 |
| Commentary     | This slide is about Why Textures Matter. Read it as sampled data access. Identify coordinates, texture object/state, filtering, mip level, and how the shader interprets the sampled value. The visible cue is: ■ Geometry defines shape / ■ Textures define appearance detail / ■ This separates visual complexity from geometric complexity / ■ Instead of modeling every scratch, brick, pore, label, or color variation, we store it in texture data / ■ Textures are efficient |
| Why it matters | Texture sampling is a major source of visual detail and a common source of artifacts.                                                                                                                                                                                                                                                                                                                                                                                               |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Check yourself | Can you identify the sampled data, coordinate, and filtering/state issue in 'Why Textures Matter'?                                                                                                                                                                                                                                                                                                                                                                  |
| Page 6         | Textures Are More Than Color Images                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Source cue     | ■ Modern rendering usually uses several texture maps per object / ■ Each map controls one part of the material or shading model / ■ Textures are therefore general-purpose GPU data sources / ■ Common texture maps / • Base color / albedo                                                                                                                                                                                                                         |
| Commentary     | This slide is about Textures Are More Than Color Images. Read it as local shading. Identify normal, light direction, view direction, material coefficients, and where the computation is evaluated. The visible cue is: ■ Modern rendering usually uses several texture maps per object / ■ Each map controls one part of the material or shading model / ■ Textures are therefore general-purpose GPU data sources / ■ Common texture maps / • Base color / albedo |
| Why it matters | Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.                                                                                                                                                                                                                                                                                                                                                              |
| Check yourself | Can you identify the normal, light vector, view vector, and material term relevant to 'Textures Are More Than Color Images'?                                                                                                                                                                                                                                                                                                                                        |
| Page 7         | Texturing Demo                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| Source cue     | Preset Plain Object / Rotate / Base color / albedo / Normal / Roughness                                                                                                                                                                                                                                                                                                                                                                                             |
| Commentary     | This slide is about Texturing Demo. Read it as local shading. Identify normal, light direction, view direction, material coefficients, and where the computation is evaluated. The visible cue is: Preset Plain Object / Rotate / Base color / albedo / Normal / Roughness                                                                                                                                                                                          |
| Why it matters | Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.                                                                                                                                                                                                                                                                                                                                                              |
| Check yourself | Can you identify the normal, light vector, view vector, and material term relevant to 'Texturing Demo'?                                                                                                                                                                                                                                                                                                                                                             |
| Page 8         | Overview                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| Source cue     | 9.1 Texture Objects, Texels, Formats, and Color Space / 9.2 Texture Coordinates, UV Mapping, and Wrapping / 9.3 Sampling, Filtering, Mipmaps, and Anisotropy / 9.4 Modern OpenGL Texture Pipeline / 9.5 Normal Mapping                                                                                                                                                                                                                                              |
| Commentary     | This slide is about Overview. Read it as concrete API state and GPU data movement. Ask which object is bound, which shader stage consumes it, and which state affects the draw call. The visible cue is: 9.1 Texture Objects, Texels, Formats, and Color Space / 9.2 Texture Coordinates, UV Mapping, and Wrapping / 9.3 Sampling, Filtering, Mipmaps, and Anisotropy / 9.4 Modern OpenGL Texture Pipeline / 9.5 Normal Mapping                                     |
| Why it matters | OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.                                                                                                                                                                                                                                                                                                                                                             |
| Check yourself | Can you name the OpenGL object, state, shader stage, or buffer involved in 'Overview'?                                                                                                                                                                                                                                                                                                                                                                              |
| Page 9         | 9.1 Texture Objects, Texels, Formats, and Color Space                                                                                                                                                                                                                                                                                                                                                                                                               |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Source cue     | What texture data stores and how the GPU interprets it                                                                                                                                                                                                                                                                                                                                                                                                  |
| Commentary     | This slide is about 9.1 Texture Objects, Texels, Formats, and Color Space. Read it as sampled data access. Identify coordinates, texture object/state, filtering, mip level, and how the shader interprets the sampled value. The visible cue is: What texture data stores and how the GPU interprets it                                                                                                                                                |
| Why it matters | Texture sampling is a major source of visual detail and a common source of artifacts.                                                                                                                                                                                                                                                                                                                                                                   |
| Check yourself | Can you identify the sampled data, coordinate, and filtering/state issue in '9.1 Texture Objects, Texels, Formats, and Color Space'?                                                                                                                                                                                                                                                                                                                    |
| Page 10        | From Image File to GPU Texture Object                                                                                                                                                                                                                                                                                                                                                                                                                   |
| Source cue     | ■ Texture data starts as image file or generated array / ■ During loading, data is uploaded to GPU memory / ■ On the GPU, data becomes a texture object / ■ The shader accesses it through a sampler / ■ Pipeline                                                                                                                                                                                                                                       |
| Commentary     | This slide is about From Image File to GPU Texture Object. Read it as concrete API state and GPU data movement. Ask which object is bound, which shader stage consumes it, and which state affects the draw call. The visible cue is: ■ Texture data starts as image file or generated array / ■ During loading, data is uploaded to GPU memory / ■ On the GPU, data becomes a texture object / ■ The shader accesses it through a sampler / ■ Pipeline |
| Why it matters | OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.                                                                                                                                                                                                                                                                                                                                                 |
| Check yourself | Can you name the OpenGL object, state, shader stage, or buffer involved in 'From Image File to GPU Texture Object'?                                                                                                                                                                                                                                                                                                                                     |
| Page 11        | Texels, Channels, and Meaning                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Source cue     | ■ Texel: smallest sample element of a texture / ■ In 2D, a texel is analogous to a pixel / ■ Texels store numeric channel values / ■ Common channel layouts / • R, RG, RGB, RGBA                                                                                                                                                                                                                                                                        |
| Commentary     | This slide is about Texels, Channels, and Meaning. Read it as sampled data access. Identify coordinates, texture object/state, filtering, mip level, and how the shader interprets the sampled value. The visible cue is: ■ Texel: smallest sample element of a texture / ■ In 2D, a texel is analogous to a pixel / ■ Texels store numeric channel values / ■ Common channel layouts / • R, RG, RGB, RGBA                                              |
| Why it matters | Texture sampling is a major source of visual detail and a common source of artifacts.                                                                                                                                                                                                                                                                                                                                                                   |
| Check yourself | Can you identify the sampled data, coordinate, and filtering/state issue in 'Texels, Channels, and Meaning'?                                                                                                                                                                                                                                                                                                                                            |
| Page 12        | Texture Targets and Dimensionality                                                                                                                                                                                                                                                                                                                                                                                                                      |
| Source cue     | ■ Texture target describes the sampled data domain / ■ Different targets use different coordinate types / ■ 1D texture / • Coordinate / • Lookup tables or transfer functions                                                                                                                                                                                                                                                                           |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                     |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Commentary     | This slide is about Texture Targets and Dimensionality. Read it as a coordinate-space operation. Name the input space, the matrix or transformation, and the output space before memorizing formulas. The visible cue is: ■ Texture target describes the sampled data domain / ■ Different targets use different coordinate types / ■ 1D texture / • Coordinate / • Lookup tables or transfer functions             |
| Why it matters | A wrong coordinate-space assumption can make correct formulas produce wrong images.                                                                                                                                                                                                                                                                                                                                 |
| Check yourself | Can you state the coordinate space before and after 'Texture Targets and Dimensionality'?                                                                                                                                                                                                                                                                                                                           |
| Page 13        | Internal Texture Formats                                                                                                                                                                                                                                                                                                                                                                                            |
| Source cue     | ■ Internal format defines GPU texel storage / ■ It specifies channels, precision, and representation / ■ Examples / • GL_R8: one 8-bit normalized channel / • GL_RGBA8: four 8-bit normalized channels                                                                                                                                                                                                              |
| Commentary     | This slide is about Internal Texture Formats. Read it as local shading. Identify normal, light direction, view direction, material coefficients, and where the computation is evaluated. The visible cue is: ■ Internal format defines GPU texel storage / ■ It specifies channels, precision, and representation / ■ Examples / • GL_R8: one 8-bit normalized channel / • GL_RGBA8: four 8-bit normalized channels |
| Why it matters | Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.                                                                                                                                                                                                                                                                                                              |
| Check yourself | Can you identify the normal, light vector, view vector, and material term relevant to 'Internal Texture Formats'?                                                                                                                                                                                                                                                                                                   |
| Page 14        | Numeric Texture Representations                                                                                                                                                                                                                                                                                                                                                                                     |
| Source cue     | ■ Texel values can be interpreted differently / ■ Normalized integer formats / • Stored as integers / • Sampled as normalized floats / 255 → 1.0                                                                                                                                                                                                                                                                    |
| Commentary     | This slide is about Numeric Texture Representations. Read it as local shading. Identify normal, light direction, view direction, material coefficients, and where the computation is evaluated. The visible cue is: ■ Texel values can be interpreted differently / ■ Normalized integer formats / • Stored as integers / • Sampled as normalized floats / 255 → 1.0                                                |
| Why it matters | Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.                                                                                                                                                                                                                                                                                                              |
| Check yourself | Can you identify the normal, light vector, view vector, and material term relevant to 'Numeric Texture Representations'?                                                                                                                                                                                                                                                                                            |
| Page 15        | Color Textures vs. Data Textures                                                                                                                                                                                                                                                                                                                                                                                    |
| Source cue     | ■ Some textures store perceptual color / ■ Other textures store numeric shader data / ■ Color textures / • Base color / albedo / • Emissive color                                                                                                                                                                                                                                                                   |
| Commentary     | This slide is about Color Textures vs. Data Textures. Read it as concrete API state and GPU data movement. Ask which object is bound, which shader stage consumes it, and which state affects the draw call. The visible cue is: ■ Some textures store perceptual color / ■ Other textures store numeric shader data / ■ Color textures / • Base color / albedo / • Emissive color                                  |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                             |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Why it matters | OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.                                                                                                                                                                                                                                                                                                                     |
| Check yourself | Can you name the OpenGL object, state, shader stage, or buffer involved in 'Color Textures vs. Data Textures'?                                                                                                                                                                                                                                                                                                              |
| Page 16        | 2D Texture Access                                                                                                                                                                                                                                                                                                                                                                                                           |
| Source cue     | ■ For each fragment, texturecoordinates are interpolated / ■ During texture access texel values are bilinearly interpolated                                                                                                                                                                                                                                                                                                 |
| Commentary     | This slide is about 2D Texture Access. Read it as a coordinate-space operation. Name the input space, the matrix or transformation, and the output space before memorizing formulas. The visible cue is: ■ For each fragment, texturecoordinates are interpolated / ■ During texture access texel values are bilinearly interpolated                                                                                        |
| Why it matters | A wrong coordinate-space assumption can make correct formulas produce wrong images.                                                                                                                                                                                                                                                                                                                                         |
| Check yourself | Can you state the coordinate space before and after '2D Texture Access'?                                                                                                                                                                                                                                                                                                                                                    |
| Page 17        | Bilinear Interpolation                                                                                                                                                                                                                                                                                                                                                                                                      |
| Source cue     | ■ Example: linear interpolation in 2D (=bilinear) / $f = f(s, t)$ / ■ Interpolation requires three computations ( Notation: $i_j$ ) / $s-s / f = (1 - \alpha) \cdot f + \alpha \cdot f \alpha =$                                                                                                                                                                                                                            |
| Commentary     | This slide is about Bilinear Interpolation. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: ■ Example: linear interpolation in 2D (=bilinear) / $f = f(s, t)$ / ■ Interpolation requires three computations ( Notation: $i_j$ ) / $s-s / f = (1 - \alpha) \cdot f + \alpha \cdot f \alpha =$ |
| Why it matters | Even a sparse slide usually names a relation you must be able to explain in words.                                                                                                                                                                                                                                                                                                                                          |
| Check yourself | Can you turn 'Bilinear Interpolation' into a causal sentence instead of repeating the slide title?                                                                                                                                                                                                                                                                                                                          |
| Page 18        | Untitled slide                                                                                                                                                                                                                                                                                                                                                                                                              |
| Source cue     | The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image.                                                                                                                                                                                                                                                                                                                  |
| Commentary     | This slide is about Untitled slide. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image.                                                                                               |
| Why it matters | Even a sparse slide usually names a relation you must be able to explain in words.                                                                                                                                                                                                                                                                                                                                          |
| Check yourself | Can you turn 'Untitled slide' into a causal sentence instead of repeating the slide title?                                                                                                                                                                                                                                                                                                                                  |
| Page 19        | Texture Memory Footprint                                                                                                                                                                                                                                                                                                                                                                                                    |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Source cue     | <p>■ Texture memory depends on resolution and format / <math>\text{memory} = \text{width} \cdot \text{height} \cdot \text{bytes per texel}</math> / ■ Example memory costs / • RGBA8: 4 MiB / • RGBA8: 16 MiB</p>                                                                                                                                                                                                                                                                                    |
| Commentary     | <p>This slide is about Texture Memory Footprint. Read it as sampled data access. Identify coordinates, texture object/state, filtering, mip level, and how the shader interprets the sampled value. The visible cue is: ■ Texture memory depends on resolution and format / <math>\text{memory} = \text{width} \cdot \text{height} \cdot \text{bytes per texel}</math> / ■ Example memory costs / • RGBA8: 4 MiB / • RGBA8: 16 MiB</p>                                                               |
| Why it matters | <p>Texture sampling is a major source of visual detail and a common source of artifacts.</p>                                                                                                                                                                                                                                                                                                                                                                                                         |
| Check yourself | <p>Can you identify the sampled data, coordinate, and filtering/state issue in 'Texture Memory Footprint'?</p>                                                                                                                                                                                                                                                                                                                                                                                       |
| Page 20        | Texture Compression and Asset Pipelines                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Source cue     | <p>■ File formats and GPU formats are different concepts / ■ File formats / • PNG, JPEG, OpenEXR / • Used for storage and authoring / ■ GPU formats</p>                                                                                                                                                                                                                                                                                                                                              |
| Commentary     | <p>This slide is about Texture Compression and Asset Pipelines. Read it as sampled data access. Identify coordinates, texture object/state, filtering, mip level, and how the shader interprets the sampled value. The visible cue is: ■ File formats and GPU formats are different concepts / ■ File formats / • PNG, JPEG, OpenEXR / • Used for storage and authoring / ■ GPU formats</p>                                                                                                          |
| Why it matters | <p>Texture sampling is a major source of visual detail and a common source of artifacts.</p>                                                                                                                                                                                                                                                                                                                                                                                                         |
| Check yourself | <p>Can you identify the sampled data, coordinate, and filtering/state issue in 'Texture Compression and Asset Pipelines'?</p>                                                                                                                                                                                                                                                                                                                                                                        |
| Page 21        | Steps Involved in Texturing                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Source cue     | <p>■ Provide textures (e.g., load from file) / ■ Configure texture mapping(e.g., border handling, texture coordinates, texture filtering) / ■ Rendering / • Fetch texture values (usually by exploiting interpolation) / • Modify fragment color based on fetched texel value</p>                                                                                                                                                                                                                    |
| Commentary     | <p>This slide is about Steps Involved in Texturing. Read it as a coordinate-space operation. Name the input space, the matrix or transformation, and the output space before memorizing formulas. The visible cue is: ■ Provide textures (e.g., load from file) / ■ Configure texture mapping(e.g., border handling, texture coordinates, texture filtering) / ■ Rendering / • Fetch texture values (usually by exploiting interpolation) / • Modify fragment color based on fetched texel value</p> |
| Why it matters | <p>A wrong coordinate-space assumption can make correct formulas produce wrong images.</p>                                                                                                                                                                                                                                                                                                                                                                                                           |
| Check yourself | <p>Can you state the coordinate space before and after 'Steps Involved in Texturing'?</p>                                                                                                                                                                                                                                                                                                                                                                                                            |
| Page 22        | 9.2 Texture Coordinates, UV Mapping, and Wrapping                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| Source cue     | Aligning texture space with geometry                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |

|                |                                                                                                                                                                                                                                                                               |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Commentary     | This slide is about 9.2 Texture Coordinates, UV Mapping, and Wrapping. Read it as a coordinate-space operation. Name the input space, the matrix or transformation, and the output space before memorizing formulas. The visible cue is: Aligning texture space with geometry |
| Why it matters | A wrong coordinate-space assumption can make correct formulas produce wrong images.                                                                                                                                                                                           |
| Check yourself | Can you state the coordinate space before and after '9.2 Texture Coordinates, UV Mapping, and Wrapping'?                                                                                                                                                                      |

|                |                                                                                                                                                                                                                                                                                                                                                                                               |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 23        | Texture Coordinate System 1/2                                                                                                                                                                                                                                                                                                                                                                 |
| Source cue     | ■ Object space: 3D vertices in model coordinate system / $(x, y, z)$ / • Coordinate representation: / ■ Texture space: 1D, 2D, 3D space of texture data / $(s, t, r, q)$                                                                                                                                                                                                                      |
| Commentary     | This slide is about Texture Coordinate System 1/2. Read it as a coordinate-space operation. Name the input space, the matrix or transformation, and the output space before memorizing formulas. The visible cue is: ■ Object space: 3D vertices in model coordinate system / $(x, y, z)$ / • Coordinate representation: / ■ Texture space: 1D, 2D, 3D space of texture data / $(s, t, r, q)$ |
| Why it matters | A wrong coordinate-space assumption can make correct formulas produce wrong images.                                                                                                                                                                                                                                                                                                           |
| Check yourself | Can you state the coordinate space before and after 'Texture Coordinate System 1/2'?                                                                                                                                                                                                                                                                                                          |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 24        | Texture Coordinate System 2/2                                                                                                                                                                                                                                                                                                                                                                                  |
| Source cue     | $[0, 1]^2$ / ■ Texture coordinate system normalized to for 2D textures (analog for 1D and 3D textures) / $s, t, r, q$ $s, t, r, q \in [0, 1]$ / • Coordinate axes denoted by with / $(s)$                                                                                                                                                                                                                      |
| Commentary     | This slide is about Texture Coordinate System 2/2. Read it as a coordinate-space operation. Name the input space, the matrix or transformation, and the output space before memorizing formulas. The visible cue is: $[0, 1]^2$ / ■ Texture coordinate system normalized to for 2D textures (analog for 1D and 3D textures) / $s, t, r, q$ $s, t, r, q \in [0, 1]$ / • Coordinate axes denoted by with / $(s)$ |
| Why it matters | A wrong coordinate-space assumption can make correct formulas produce wrong images.                                                                                                                                                                                                                                                                                                                            |
| Check yourself | Can you state the coordinate space before and after 'Texture Coordinate System 2/2'?                                                                                                                                                                                                                                                                                                                           |

|            |                                                                                                                                                                                                                                                                                                         |
|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 25    | Texture Coordinate Specification                                                                                                                                                                                                                                                                        |
| Source cue | ■ 2D texturing / $p = (x, y, z)$ $p = (s, t)$ / • For every vertex $g$ of a polygon, texture coordinates $t$ are specified / • Texture coordinates $t$ are obtained from parametrization algorithms / • Texture coordinates $t$ are linearly interpolated over a polygon (analogous to Gouraud shading) |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Commentary     | This slide is about Texture Coordinate Specification. Read it as a coordinate-space operation. Name the input space, the matrix or transformation, and the output space before memorizing formulas. The visible cue is: <b>■</b> 2D texturing / $p = (x, y, z)$ $p = (s, t)$ / • For every vertex $g$ of a polygon, texture coordinates $t$ are specified / • Texture coordinates $t$ are obtained from parametrization algorithms / • Texture coordinates $t$ are linearly interpolated over a polygon (analogous to Gouraud shading) |
| Why it matters | A wrong coordinate-space assumption can make correct formulas produce wrong images.                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| Check yourself | Can you state the coordinate space before and after 'Texture Coordinate Specification'?                                                                                                                                                                                                                                                                                                                                                                                                                                                |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 26        | Disc Parametrization                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Source cue     | $r$ $xy$ / <b>■</b> Circular disc with radius , centered in the plane / $p = (x, y, z) = (a \cdot \cos(\theta), a \cdot \sin(\theta), 0)$ $\theta \in [0, 2\pi]$ $a \in [0, r]$ / • $g$ with and / $p = (s, t) \in [0, 1]^2$ $s = \theta$ $t = a$                                                                                                                                                                                                                          |
| Commentary     | This slide is about Disc Parametrization. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: $r$ $xy$ / <b>■</b> Circular disc with radius , centered in the plane / $p = (x, y, z) = (a \cdot \cos(\theta), a \cdot \sin(\theta), 0)$ $\theta \in [0, 2\pi]$ $a \in [0, r]$ / • $g$ with and / $p = (s, t) \in [0, 1]^2$ $s = \theta$ $t = a$ |
| Why it matters | Even a sparse slide usually names a relation you must be able to explain in words.                                                                                                                                                                                                                                                                                                                                                                                         |
| Check yourself | Can you turn 'Disc Parametrization' into a causal sentence instead of repeating the slide title?                                                                                                                                                                                                                                                                                                                                                                           |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 27        | Cylinder Parametrization                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Source cue     | $xy$ $r = 1$ $h$ $z$ / <b>■</b> Cylinder based in plane with radius and height along axis (surface is topologically / two-dimensional and overlap-free) / $p = (x, y, z) = (r \cdot \cos(\theta), r \cdot \sin(\theta), z)$ $\theta \in [0, 2\pi]$ $z \in [0, h]$ / • $g$ with and                                                                                                                                                                                                                              |
| Commentary     | This slide is about Cylinder Parametrization. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: $xy$ $r = 1$ $h$ $z$ / <b>■</b> Cylinder based in plane with radius and height along axis (surface is topologically / two-dimensional and overlap-free) / $p = (x, y, z) = (r \cdot \cos(\theta), r \cdot \sin(\theta), z)$ $\theta \in [0, 2\pi]$ $z \in [0, h]$ / • $g$ with and |
| Why it matters | Even a sparse slide usually names a relation you must be able to explain in words.                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Check yourself | Can you turn 'Cylinder Parametrization' into a causal sentence instead of repeating the slide title?                                                                                                                                                                                                                                                                                                                                                                                                            |

|            |                                                                                                                                                                                                                                                                 |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 28    | Two-Part Mapping 1/2                                                                                                                                                                                                                                            |
| Source cue | <b>■</b> Texture coordinates for complex geometries are derived from intermediate objects (i.e., geometries / with inherent parametrization) / <b>■</b> Proceeding / • Calculate texture coordinates for intermediate object / $p = (x, y, z)$ $p = (s, t)$ $p$ |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Commentary     | This slide is about Two-Part Mapping 1/2. Read it as a coordinate-space operation. Name the input space, the matrix or transformation, and the output space before memorizing formulas. The visible cue is: ■ Texture coordinates for complex geometries are derived from intermediate objects (i.e., geometries / with inherent parametrization) / ■ Proceeding / • Calculate texture coordinates for intermediate object / $p = (x, y, z)$ $p = (s, t)$ $p$ |
| Why it matters | A wrong coordinate-space assumption can make correct formulas produce wrong images.                                                                                                                                                                                                                                                                                                                                                                           |
| Check yourself | Can you state the coordinate space before and after 'Two-Part Mapping 1/2'?                                                                                                                                                                                                                                                                                                                                                                                   |

|                |                                                                                                                                                                                                                                                                                                                                                           |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 29        | Two-Part Mapping 2/2                                                                                                                                                                                                                                                                                                                                      |
| Source cue     | ■ Two-part mapping has no restrictions wrt. objects to be textured / ■ Intermediate object must be appropriate for object to be textured                                                                                                                                                                                                                  |
| Commentary     | This slide is about Two-Part Mapping 2/2. Read it as sampled data access. Identify coordinates, texture object/state, filtering, mip level, and how the shader interprets the sampled value. The visible cue is: ■ Two-part mapping has no restrictions wrt. objects to be textured / ■ Intermediate object must be appropriate for object to be textured |
| Why it matters | Texture sampling is a major source of visual detail and a common source of artifacts.                                                                                                                                                                                                                                                                     |
| Check yourself | Can you identify the sampled data, coordinate, and filtering/state issue in 'Two-Part Mapping 2/2'?                                                                                                                                                                                                                                                       |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 30        | Two-Part Mapping Example                                                                                                                                                                                                                                                                                                                                                                                                                      |
| Source cue     | ■ Object: non-convex polyhedron / ■ Intermediate object: cylinder / • 1 : Cast rays from object center through vertices to intersect cylinder (projection) / • 2 : Cylinder parametrization / $F(p) = F(F(p)) = (s, t)$                                                                                                                                                                                                                       |
| Commentary     | This slide is about Two-Part Mapping Example. Read it as camera geometry. Track how 3D view-space positions become clip coordinates, normalized device coordinates, and finally screen locations. The visible cue is: ■ Object: non-convex polyhedron / ■ Intermediate object: cylinder / • 1 : Cast rays from object center through vertices to intersect cylinder (projection) / • 2 : Cylinder parametrization / $F(p) = F(F(p)) = (s, t)$ |
| Why it matters | Projection controls both image composition and depth precision, so it affects visibility and rasterization later.                                                                                                                                                                                                                                                                                                                             |
| Check yourself | Can you explain how 'Two-Part Mapping Example' changes positions before rasterization?                                                                                                                                                                                                                                                                                                                                                        |

|            |                                                                                                                                                                                                                                                                                                           |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 31    | Projection Calculation Variations                                                                                                                                                                                                                                                                         |
| Source cue | O O / ■ G center through vertices to I / O O / ■ G along normal to I / O O                                                                                                                                                                                                                                |
| Commentary | This slide is about Projection Calculation Variations. Read it as camera geometry. Track how 3D view-space positions become clip coordinates, normalized device coordinates, and finally screen locations. The visible cue is: O O / ■ G center through vertices to I / O O / ■ G along normal to I / O O |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Why it matters | Projection controls both image composition and depth precision, so it affects visibility and rasterization later.                                                                                                                                                                                                                                                                                                                               |
| Check yourself | Can you explain how 'Projection Calculation Variations' changes positions before rasterization?                                                                                                                                                                                                                                                                                                                                                 |
| Page 32        | Texture Coordinate Transformation                                                                                                                                                                                                                                                                                                                                                                                                               |
| Source cue     | <ul style="list-style-type: none"> <li>■ Texture coordinates can be transformed through transformation matrix /</li> <li>• No restriction regarding matrix coefficients (4x4 matrix) / ■ Applications /</li> <li>• Shifting texture coordinates / • Scaling texture coordinates</li> </ul>                                                                                                                                                      |
| Commentary     | This slide is about Texture Coordinate Transformation. Read it as a coordinate-space operation. Name the input space, the matrix or transformation, and the output space before memorizing formulas. The visible cue is: ■ Texture coordinates can be transformed through transformation matrix / • No restriction regarding matrix coefficients (4x4 matrix) / ■ Applications / • Shifting texture coordinates / • Scaling texture coordinates |
| Why it matters | A wrong coordinate-space assumption can make correct formulas produce wrong images.                                                                                                                                                                                                                                                                                                                                                             |
| Check yourself | Can you state the coordinate space before and after 'Texture Coordinate Transformation'?                                                                                                                                                                                                                                                                                                                                                        |
| Page 33        | Demo - Texture Coordinate Transformation                                                                                                                                                                                                                                                                                                                                                                                                        |
| Source cue     | Stage Transformed Texture / scaleS 1 / scaleT 1 / offsetS 0 / offsetT 0                                                                                                                                                                                                                                                                                                                                                                         |
| Commentary     | This slide is about Demo - Texture Coordinate Transformation. Read it as a coordinate-space operation. Name the input space, the matrix or transformation, and the output space before memorizing formulas. The visible cue is: Stage Transformed Texture / scaleS 1 / scaleT 1 / offsetS 0 / offsetT 0                                                                                                                                         |
| Why it matters | A wrong coordinate-space assumption can make correct formulas produce wrong images.                                                                                                                                                                                                                                                                                                                                                             |
| Check yourself | Can you state the coordinate space before and after 'Demo - Texture Coordinate Transformation'?                                                                                                                                                                                                                                                                                                                                                 |
| Page 34        | Height Field Texturing Example                                                                                                                                                                                                                                                                                                                                                                                                                  |
| Source cue     | <ul style="list-style-type: none"> <li>■ Height field represented as triangular mesh / (x, y) (s, t) / • Use normalized coordinates as texture coordinates (can be used to map aerial / image) / z r</li> </ul>                                                                                                                                                                                                                                 |
| Commentary     | This slide is about Height Field Texturing Example. Read it as a coordinate-space operation. Name the input space, the matrix or transformation, and the output space before memorizing formulas. The visible cue is: ■ Height field represented as triangular mesh / (x, y) (s, t) / • Use normalized coordinates as texture coordinates (can be used to map aerial / image) / z r                                                             |
| Why it matters | A wrong coordinate-space assumption can make correct formulas produce wrong images.                                                                                                                                                                                                                                                                                                                                                             |
| Check yourself | Can you state the coordinate space before and after 'Height Field Texturing Example'?                                                                                                                                                                                                                                                                                                                                                           |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 35        | 9.3 Sampling, Filtering, Mipmaps, and Anisotropy                                                                                                                                                                                                                                                                                                                                                                                                                            |
| Source cue     | Reconstructing texture values under magnification and minification                                                                                                                                                                                                                                                                                                                                                                                                          |
| Commentary     | This slide is about 9.3 Sampling, Filtering, Mipmaps, and Anisotropy. Read it as sampled data access. Identify coordinates, texture object/state, filtering, mip level, and how the shader interprets the sampled value. The visible cue is: Reconstructing texture values under magnification and minification                                                                                                                                                             |
| Why it matters | Texture sampling is a major source of visual detail and a common source of artifacts.                                                                                                                                                                                                                                                                                                                                                                                       |
| Check yourself | Can you identify the sampled data, coordinate, and filtering/state issue in '9.3 Sampling, Filtering, Mipmaps, and Anisotropy'?                                                                                                                                                                                                                                                                                                                                             |
| Page 36        | Forward vs. Inverse Mapping                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Source cue     | ■ Forward mapping - from texture to screen space / • Object's surface parameterization / • Projection transformation / ■ Inverse mapping - from screen to texture space / • Find corresponding pre-image/footprint of each pixel in texture                                                                                                                                                                                                                                 |
| Commentary     | This slide is about Forward vs. Inverse Mapping. Read it as a coordinate-space operation. Name the input space, the matrix or transformation, and the output space before memorizing formulas. The visible cue is: ■ Forward mapping - from texture to screen space / • Object's surface parameterization / • Projection transformation / ■ Inverse mapping - from screen to texture space / • Find corresponding pre-image/footprint of each pixel in texture              |
| Why it matters | A wrong coordinate-space assumption can make correct formulas produce wrong images.                                                                                                                                                                                                                                                                                                                                                                                         |
| Check yourself | Can you state the coordinate space before and after 'Forward vs. Inverse Mapping'?                                                                                                                                                                                                                                                                                                                                                                                          |
| Page 37        | Forward Mapping                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| Source cue     | ■ Maps each texel to screen space / • Uniform sampling of texture does not guarantee uniform sampling in screen space / ■ Forward mapping used for following cases / • Texture-to-screen mapping difficult to invert / • Texture image does not fit into memory                                                                                                                                                                                                             |
| Commentary     | This slide is about Forward Mapping. Read it as sampled data access. Identify coordinates, texture object/state, filtering, mip level, and how the shader interprets the sampled value. The visible cue is: ■ Maps each texel to screen space / • Uniform sampling of texture does not guarantee uniform sampling in screen space / ■ Forward mapping used for following cases / • Texture-to-screen mapping difficult to invert / • Texture image does not fit into memory |
| Why it matters | Texture sampling is a major source of visual detail and a common source of artifacts.                                                                                                                                                                                                                                                                                                                                                                                       |
| Check yourself | Can you identify the sampled data, coordinate, and filtering/state issue in 'Forward Mapping'?                                                                                                                                                                                                                                                                                                                                                                              |
| Page 38        | Inverse Mapping 1/2                                                                                                                                                                                                                                                                                                                                                                                                                                                         |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Source cue     | ■ For each pixel pre-image in texture space is found and its area is integrated over / ■ Inverse mapping can be used when / • Texture image fits into memory / • Forward mapping can be inverted / 1 for y in range(height): # screen 'height'                                                                                                                                                                                                                 |
| Commentary     | This slide is about Inverse Mapping 1/2. Read it as sampled data access. Identify coordinates, texture object/state, filtering, mip level, and how the shader interprets the sampled value. The visible cue is: ■ For each pixel pre-image in texture space is found and its area is integrated over / ■ Inverse mapping can be used when / • Texture image fits into memory / • Forward mapping can be inverted / 1 for y in range(height): # screen 'height' |
| Why it matters | Texture sampling is a major source of visual detail and a common source of artifacts.                                                                                                                                                                                                                                                                                                                                                                          |
| Check yourself | Can you identify the sampled data, coordinate, and filtering/state issue in 'Inverse Mapping 1/2'?                                                                                                                                                                                                                                                                                                                                                             |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 39        | Inverse Mapping 2/2                                                                                                                                                                                                                                                                                                                                                                                                               |
| Source cue     | ■ Pre-image of square screen pixel intersecting curved surface is curvilinear quadrilateral in texture / space / ■ Curvilinear quadrilateral often approximated / • By general quadrilateral / • By parallelogram                                                                                                                                                                                                                 |
| Commentary     | This slide is about Inverse Mapping 2/2. Read it as sampled data access. Identify coordinates, texture object/state, filtering, mip level, and how the shader interprets the sampled value. The visible cue is: ■ Pre-image of square screen pixel intersecting curved surface is curvilinear quadrilateral in texture / space / ■ Curvilinear quadrilateral often approximated / • By general quadrilateral / • By parallelogram |
| Why it matters | Texture sampling is a major source of visual detail and a common source of artifacts.                                                                                                                                                                                                                                                                                                                                             |
| Check yourself | Can you identify the sampled data, coordinate, and filtering/state issue in 'Inverse Mapping 2/2'?                                                                                                                                                                                                                                                                                                                                |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                    |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 40        | Magnification vs. Minification                                                                                                                                                                                                                                                                                                                                                                                     |
| Source cue     | ■ Magnification / • Map few texels onto many pixels / • Two filtering approaches / • Nearest: Take the nearest texel / • Bilinear interpolation: Interpolation between 4 nearest texels                                                                                                                                                                                                                            |
| Commentary     | This slide is about Magnification vs. Minification. Read it as sampled data access. Identify coordinates, texture object/state, filtering, mip level, and how the shader interprets the sampled value. The visible cue is: ■ Magnification / • Map few texels onto many pixels / • Two filtering approaches / • Nearest: Take the nearest texel / • Bilinear interpolation: Interpolation between 4 nearest texels |
| Why it matters | Texture sampling is a major source of visual detail and a common source of artifacts.                                                                                                                                                                                                                                                                                                                              |
| Check yourself | Can you identify the sampled data, coordinate, and filtering/state issue in 'Magnification vs. Minification'?                                                                                                                                                                                                                                                                                                      |

|         |                                  |
|---------|----------------------------------|
| Page 41 | Nearest vs. Linear Interpolation |
|---------|----------------------------------|

|                |                                                                                                                                                                                                                                                                                                                                                                             |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Source cue     | ■ Nearest neighbor interpolation / (s, t) / • For coordinates select nearest integer texel / (int(round(s · texdim)), int(round(t · texdim))) / x y                                                                                                                                                                                                                         |
| Commentary     | This slide is about Nearest vs. Linear Interpolation. Read it as a coordinate-space operation. Name the input space, the matrix or transformation, and the output space before memorizing formulas. The visible cue is: ■ Nearest neighbor interpolation / (s, t) / • For coordinates select nearest integer texel / (int(round(s · texdim)), int(round(t · texdim))) / x y |
| Why it matters | A wrong coordinate-space assumption can make correct formulas produce wrong images.                                                                                                                                                                                                                                                                                         |
| Check yourself | Can you state the coordinate space before and after 'Nearest vs. Linear Interpolation'?                                                                                                                                                                                                                                                                                     |
| Page 42        | Minification Filtering Requirements                                                                                                                                                                                                                                                                                                                                         |
| Source cue     | ■ Preimage integration with insufficient filtering results in aliasing / artifacts                                                                                                                                                                                                                                                                                          |
| Commentary     | This slide is about Minification Filtering Requirements. Read it as sampled data access. Identify coordinates, texture object/state, filtering, mip level, and how the shader interprets the sampled value. The visible cue is: ■ Preimage integration with insufficient filtering results in aliasing / artifacts                                                          |
| Why it matters | Texture sampling is a major source of visual detail and a common source of artifacts.                                                                                                                                                                                                                                                                                       |
| Check yourself | Can you identify the sampled data, coordinate, and filtering/state issue in 'Minification Filtering Requirements'?                                                                                                                                                                                                                                                          |
| Page 43        | Demo - Magnification Filtering: Nearest vs Linear                                                                                                                                                                                                                                                                                                                           |
| Source cue     | Stage Split Comparison / magFilter GL_NEAREST / zoom 12 / textureResolution 8x8 / showTexelGrid                                                                                                                                                                                                                                                                             |
| Commentary     | This slide is about Demo - Magnification Filtering: Nearest vs Linear. Read it as sampled data access. Identify coordinates, texture object/state, filtering, mip level, and how the shader interprets the sampled value. The visible cue is: Stage Split Comparison / magFilter GL_NEAREST / zoom 12 / textureResolution 8x8 / showTexelGrid                               |
| Why it matters | Texture sampling is a major source of visual detail and a common source of artifacts.                                                                                                                                                                                                                                                                                       |
| Check yourself | Can you identify the sampled data, coordinate, and filtering/state issue in 'Demo - Magnification Filtering: Nearest vs Linear'?                                                                                                                                                                                                                                            |
| Page 44        | MIP Mapping (multum in parvo = 'much in little')                                                                                                                                                                                                                                                                                                                            |
| Source cue     | ■ Efficient scheme for sampling texture values / ■ Representation of a texture at different resolution levels(levels of Detail, LoDs) / ■ Exploiting a texture pyramid (here: 2D, analog for 1D and 3D) / • Each pyramid level contains filtered texture of half resolution of previous level / • Each layer contains only quarter of texels of previous layer              |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Commentary     | This slide is about MIP Mapping (multum in parvo = 'much in little'). Read it as sampled data access. Identify coordinates, texture object/state, filtering, mip level, and how the shader interprets the sampled value. The visible cue is: ■ Efficient scheme for sampling texture values / ■ Representation of a texture at different resolution levels(levels of Detail, LoDs) / ■ Exploiting a texture pyramid (here: 2D, analog for 1D and 3D) / • Each pyramid level contains filtered texture of half resolution of previous level / • Each layer contains only quarter of texels of previous layer |
| Why it matters | Texture sampling is a major source of visual detail and a common source of artifacts.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Check yourself | Can you identify the sampled data, coordinate, and filtering/state issue in 'MIP Mapping (multum in parvo = 'much in little')'?                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| Page 45        | MIP Map Pyramid 1/2                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Source cue     | ■ MIP Map-Level 0 : Texture in original size / $2^{N-1}$ / ■ MIP Map-Level 1: Texture size / ■ ... / $N \cdot 2^0$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Commentary     | This slide is about MIP Map Pyramid 1/2. Read it as sampled data access. Identify coordinates, texture object/state, filtering, mip level, and how the shader interprets the sampled value. The visible cue is: ■ MIP Map-Level 0 : Texture in original size / $2^{N-1}$ / ■ MIP Map-Level 1: Texture size / ■ ... / $N \cdot 2^0$                                                                                                                                                                                                                                                                          |
| Why it matters | Texture sampling is a major source of visual detail and a common source of artifacts.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Check yourself | Can you identify the sampled data, coordinate, and filtering/state issue in 'MIP Map Pyramid 1/2'?                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Page 46        | MIP Map Pyramid 2/2                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Source cue     | ■ Example MIP Map / ■ Self-Contained MIP Map for efficient storage                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Commentary     | This slide is about MIP Map Pyramid 2/2. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: ■ Example MIP Map / ■ Self-Contained MIP Map for efficient storage                                                                                                                                                                                                                                                                                                                  |
| Why it matters | Even a sparse slide usually names a relation you must be able to explain in words.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Check yourself | Can you turn 'MIP Map Pyramid 2/2' into a causal sentence instead of repeating the slide title?                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| Page 47        | MIP Mapping Memory                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Source cue     | ■ 33% more memory is needed                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Commentary     | This slide is about MIP Mapping Memory. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: ■ 33% more memory is needed                                                                                                                                                                                                                                                                                                                                                          |
| Why it matters | Even a sparse slide usually names a relation you must be able to explain in words.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Check yourself | Can you turn 'MIP Mapping Memory' into a causal sentence instead of repeating the slide title?                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 48        | MIP Mapping Procedure                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| Source cue     | $\lambda \in [0, N]$ / ■ Selection of the resolution levels based on continuous LoD level for a given fragment / ■ Procedure / • LoD level calculation / • Intra-level interpolation: interpolation within two levels closest to                                                                                                                                                                                                                           |
| Commentary     | This slide is about MIP Mapping Procedure. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: $\lambda \in [0, N]$ / ■ Selection of the resolution levels based on continuous LoD level for a given fragment / ■ Procedure / • LoD level calculation / • Intra-level interpolation: interpolation within two levels closest to |
| Why it matters | Even a sparse slide usually names a relation you must be able to explain in words.                                                                                                                                                                                                                                                                                                                                                                         |
| Check yourself | Can you turn 'MIP Mapping Procedure' into a causal sentence instead of repeating the slide title?                                                                                                                                                                                                                                                                                                                                                          |
| Page 49        | LoD Calculation 1/2                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Source cue     | ■ Assumptions / $[0, 1]^2$ / • 2D texture is parametrized with texture coordinates / $M \cdot M = 2N$ / • 2D texture has texels with (power-of-two texture)                                                                                                                                                                                                                                                                                                |
| Commentary     | This slide is about LoD Calculation 1/2. Read it as a coordinate-space operation. Name the input space, the matrix or transformation, and the output space before memorizing formulas. The visible cue is: ■ Assumptions / $[0, 1]^2$ / • 2D texture is parametrized with texture coordinates / $M \cdot M = 2N$ / • 2D texture has texels with (power-of-two texture)                                                                                     |
| Why it matters | A wrong coordinate-space assumption can make correct formulas produce wrong images.                                                                                                                                                                                                                                                                                                                                                                        |
| Check yourself | Can you state the coordinate space before and after 'LoD Calculation 1/2'?                                                                                                                                                                                                                                                                                                                                                                                 |
| Page 50        | LoD Calculation 2/2                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Source cue     | ■ Approximation of the preimage by square / ■ Calculation of the MIP map level / • Area of approximating square is / $b = A$ / • Length of square's side is                                                                                                                                                                                                                                                                                                |
| Commentary     | This slide is about LoD Calculation 2/2. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: ■ Approximation of the preimage by square / ■ Calculation of the MIP map level / • Area of approximating square is / $b = A$ / • Length of square's side is                                                                        |
| Why it matters | Even a sparse slide usually names a relation you must be able to explain in words.                                                                                                                                                                                                                                                                                                                                                                         |
| Check yourself | Can you turn 'LoD Calculation 2/2' into a causal sentence instead of repeating the slide title?                                                                                                                                                                                                                                                                                                                                                            |
| Page 51        | Demo - Minification, Mipmaps, and Mipmap Level Visualization                                                                                                                                                                                                                                                                                                                                                                                               |
| Source cue     | Stage Minified Floor / minFilter GL_LINEAR_MIPMAP_LINEAR / mipmapsEnabled / lodBias 0 / visualizeMipLevels                                                                                                                                                                                                                                                                                                                                                 |

|                |                                                                                                                                                                                                                                                                                                                                                                     |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Commentary     | This slide is about Demo - Minification, Mipmaps, and Mipmap Level Visualization. Read it as sampled data access. Identify coordinates, texture object/state, filtering, mip level, and how the shader interprets the sampled value. The visible cue is: Stage Minified Floor / minFilter GL_LINEAR_MIPMAP_LINEAR / mipmapsEnabled / lodBias 0 / visualizeMipLevels |
| Why it matters | Texture sampling is a major source of visual detail and a common source of artifacts.                                                                                                                                                                                                                                                                               |
| Check yourself | Can you identify the sampled data, coordinate, and filtering/state issue in 'Demo - Minification, Mipmaps, and Mipmap Level Visualization'?                                                                                                                                                                                                                         |
| Page 52        | Anisotropic Filtering                                                                                                                                                                                                                                                                                                                                               |
| Source cue     | ■ Approximation of preimage through several squares                                                                                                                                                                                                                                                                                                                 |
| Commentary     | This slide is about Anisotropic Filtering. Read it as sampled data access. Identify coordinates, texture object/state, filtering, mip level, and how the shader interprets the sampled value. The visible cue is: ■ Approximation of preimage through several squares                                                                                               |
| Why it matters | Texture sampling is a major source of visual detail and a common source of artifacts.                                                                                                                                                                                                                                                                               |
| Check yourself | Can you identify the sampled data, coordinate, and filtering/state issue in 'Anisotropic Filtering'?                                                                                                                                                                                                                                                                |
| Page 53        | Demo - Anisotropic Filtering                                                                                                                                                                                                                                                                                                                                        |
| Source cue     | Stage 1x vs Selected Anisotropy / anisotropy 16x / minFilter GL_LINEAR_MIPMAP_LINEAR / textureFrequency 34 / cameraAngle 10                                                                                                                                                                                                                                         |
| Commentary     | This slide is about Demo - Anisotropic Filtering. Read it as camera geometry. Track how 3D view-space positions become clip coordinates, normalized device coordinates, and finally screen locations. The visible cue is: Stage 1x vs Selected Anisotropy / anisotropy 16x / minFilter GL_LINEAR_MIPMAP_LINEAR / textureFrequency 34 / cameraAngle 10               |
| Why it matters | Projection controls both image composition and depth precision, so it affects visibility and rasterization later.                                                                                                                                                                                                                                                   |
| Check yourself | Can you explain how 'Demo - Anisotropic Filtering' changes positions before rasterization?                                                                                                                                                                                                                                                                          |
| Page 54        | 9.4 Modern OpenGL Texture Pipeline                                                                                                                                                                                                                                                                                                                                  |
| Source cue     | Texture objects, sampler uniforms, and shader-based access                                                                                                                                                                                                                                                                                                          |
| Commentary     | This slide is about 9.4 Modern OpenGL Texture Pipeline. Read it as concrete API state and GPU data movement. Ask which object is bound, which shader stage consumes it, and which state affects the draw call. The visible cue is: Texture objects, sampler uniforms, and shader-based access                                                                       |
| Why it matters | OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.                                                                                                                                                                                                                                                             |
| Check yourself | Can you name the OpenGL object, state, shader stage, or buffer involved in '9.4 Modern OpenGL Texture Pipeline'?                                                                                                                                                                                                                                                    |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 55        | OpenGL Texturing                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Source cue     | ■ Texturing procedure / • Fetch interpolated texel value from texture / • Use texel value to modulate fragment color or depth / ■ Configuration / • Per texture                                                                                                                                                                                                                                                                                         |
| Commentary     | This slide is about OpenGL Texturing. Read it as concrete API state and GPU data movement. Ask which object is bound, which shader stage consumes it, and which state affects the draw call. The visible cue is: ■ Texturing procedure / • Fetch interpolated texel value from texture / • Use texel value to modulate fragment color or depth / ■ Configuration / • Per texture                                                                        |
| Why it matters | OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.                                                                                                                                                                                                                                                                                                                                                 |
| Check yourself | Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Texturing'?                                                                                                                                                                                                                                                                                                                                                          |
| Page 56        | OpenGL Minification & Magnification 1/3                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Source cue     | ■ OpenGL exposes two texture filtering parameters: / • GL_TEXTURE_MIN_FILTER / • GL_TEXTURE_MAG_FILTER / ■ Texture minification (GL_TEXTURE_MIN_FILTER) options / • GL_NEAREST - nearest neighbor interpolation                                                                                                                                                                                                                                         |
| Commentary     | This slide is about OpenGL Minification & Magnification 1/3. Read it as concrete API state and GPU data movement. Ask which object is bound, which shader stage consumes it, and which state affects the draw call. The visible cue is: ■ OpenGL exposes two texture filtering parameters: / • GL_TEXTURE_MIN_FILTER / • GL_TEXTURE_MAG_FILTER / ■ Texture minification (GL_TEXTURE_MIN_FILTER) options / • GL_NEAREST - nearest neighbor interpolation |
| Why it matters | OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.                                                                                                                                                                                                                                                                                                                                                 |
| Check yourself | Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Minification & Magnification 1/3'?                                                                                                                                                                                                                                                                                                                                   |
| Page 57        | OpenGL Minification & Magnification 2/3                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Source cue     | ■ OpenGL bilinear filtering example / 1 glEnable(GL_TEXTURE_2D); / 2 glGenTextures(1, &textureID); / 3 glBindTexture(GL_TEXTURE_2D, textureID); / 4 ...                                                                                                                                                                                                                                                                                                 |
| Commentary     | This slide is about OpenGL Minification & Magnification 2/3. Read it as concrete API state and GPU data movement. Ask which object is bound, which shader stage consumes it, and which state affects the draw call. The visible cue is: ■ OpenGL bilinear filtering example / 1 glEnable(GL_TEXTURE_2D); / 2 glGenTextures(1, &textureID); / 3 glBindTexture(GL_TEXTURE_2D, textureID); / 4 ...                                                         |
| Why it matters | OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.                                                                                                                                                                                                                                                                                                                                                 |
| Check yourself | Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Minification & Magnification 2/3'?                                                                                                                                                                                                                                                                                                                                   |
| Page 58        | OpenGL Minification & Magnification 3/3                                                                                                                                                                                                                                                                                                                                                                                                                 |

|                |                                                                                                                                                                                                                                                                                                                                                                                         |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Source cue     | ■ OpenGL mipmapping example / 1 glEnable(GL_TEXTURE_2D); / 2 glGenTextures(1, &textureID); / 3 glBindTexture(GL_TEXTURE_2D, textureID); / 4 ...                                                                                                                                                                                                                                         |
| Commentary     | This slide is about OpenGL Minification & Magnification 3/3. Read it as concrete API state and GPU data movement. Ask which object is bound, which shader stage consumes it, and which state affects the draw call. The visible cue is: ■ OpenGL mipmapping example / 1 glEnable(GL_TEXTURE_2D); / 2 glGenTextures(1, &textureID); / 3 glBindTexture(GL_TEXTURE_2D, textureID); / 4 ... |
| Why it matters | OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.                                                                                                                                                                                                                                                                                 |
| Check yourself | Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Minification & Magnification 3/3'?                                                                                                                                                                                                                                                                   |

|                |                                                                                                                                                                                                                                                                                                                                                      |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 59        | Demo - Texture Units and Sampler Uniforms                                                                                                                                                                                                                                                                                                            |
| Source cue     | Stage Combined Sampling / textureUnit0 Checker / textureUnit1 Brick / textureUnit2 UV Test / baseSamplerUnit 0                                                                                                                                                                                                                                       |
| Commentary     | This slide is about Demo - Texture Units and Sampler Uniforms. Read it as sampled data access. Identify coordinates, texture object/state, filtering, mip level, and how the shader interprets the sampled value. The visible cue is: Stage Combined Sampling / textureUnit0 Checker / textureUnit1 Brick / textureUnit2 UV Test / baseSamplerUnit 0 |
| Why it matters | Texture sampling is a major source of visual detail and a common source of artifacts.                                                                                                                                                                                                                                                                |
| Check yourself | Can you identify the sampled data, coordinate, and filtering/state issue in 'Demo - Texture Units and Sampler Uniforms'?                                                                                                                                                                                                                             |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 60        | Texture Wrapping 1/4                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Source cue     | ■ Texture wrapping specifies border handling behavior when accessing texture outside the / [0, 1]D / normalized texture value range / ■ Texture wrapping mode is part of texture specification, and specified separately for each texture / dimension                                                                                                                                                                                                          |
| Commentary     | This slide is about Texture Wrapping 1/4. Read it as local shading. Identify normal, light direction, view direction, material coefficients, and where the computation is evaluated. The visible cue is: ■ Texture wrapping specifies border handling behavior when accessing texture outside the / [0, 1]D / normalized texture value range / ■ Texture wrapping mode is part of texture specification, and specified separately for each texture / dimension |
| Why it matters | Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.                                                                                                                                                                                                                                                                                                                                                         |
| Check yourself | Can you identify the normal, light vector, view vector, and material term relevant to 'Texture Wrapping 1/4'?                                                                                                                                                                                                                                                                                                                                                  |

|            |                                                                                                                                                                             |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 61    | Texture Wrapping 2/4                                                                                                                                                        |
| Source cue | ■ GL_CLAMP_TO_EDGE / • Texture coordinates below 0 become 0 / • Texture coordinates above 1 become 1 / • Results in constant continuation of texture / ■ GL_CLAMP_TO_BORDER |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Commentary     | This slide is about Texture Wrapping 2/4. Read it as a coordinate-space operation. Name the input space, the matrix or transformation, and the output space before memorizing formulas. The visible cue is: ■<br>GL_CLAMP_TO_EDGE / • Texture coordinates below 0 become 0 / • Texture coordinates above 1 become 1 / • Results in constant continuation of texture / ■ GL_CLAMP_TO_BORDER                                                                                                                                        |
| Why it matters | A wrong coordinate-space assumption can make correct formulas produce wrong images.                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| Check yourself | Can you state the coordinate space before and after 'Texture Wrapping 2/4'?                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Page 62        | Texture Wrapping 3/4                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Source cue     | ■ GL_REPEAT / • Texture coordinates ignore digits before decimal point, and use digits after decimal point as / texture coordinates / • Results in periodic continuation of normalized texture space / ■<br>GL_MIRRORED_REPEAT                                                                                                                                                                                                                                                                                                    |
| Commentary     | This slide is about Texture Wrapping 3/4. Read it as a coordinate-space operation. Name the input space, the matrix or transformation, and the output space before memorizing formulas. The visible cue is: ■<br>GL_REPEAT / • Texture coordinates ignore digits before decimal point, and use digits after decimal point as / texture coordinates / • Results in periodic continuation of normalized texture space / ■<br>GL_MIRRORED_REPEAT                                                                                     |
| Why it matters | A wrong coordinate-space assumption can make correct formulas produce wrong images.                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| Check yourself | Can you state the coordinate space before and after 'Texture Wrapping 3/4'?                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Page 63        | Texture Wrapping 4/4                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Source cue     | ■ OpenGL code snippets / 1 glTexParameteri(textureID, GL_TEXTURE_WRAP_S, GL_MIRRORED_REPEAT); / 2 glTexParameteri(textureID, GL_TEXTURE_WRAP_T, GL_MIRRORED_REPEAT); / 1 const GLfloat borderColor[] = { 1.0f, 1.0f, 0.0f, 1.0f }; / 3 glTexParameteri(textureID, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_BORDER);                                                                                                                                                                                                                         |
| Commentary     | This slide is about Texture Wrapping 4/4. Read it as concrete API state and GPU data movement. Ask which object is bound, which shader stage consumes it, and which state affects the draw call. The visible cue is: ■<br>OpenGL code snippets / 1 glTexParameteri(textureID, GL_TEXTURE_WRAP_S, GL_MIRRORED_REPEAT); / 2 glTexParameteri(textureID, GL_TEXTURE_WRAP_T, GL_MIRRORED_REPEAT); / 1 const GLfloat borderColor[] = { 1.0f, 1.0f, 0.0f, 1.0f }; / 3 glTexParameteri(textureID, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_BORDER); |
| Why it matters | OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Check yourself | Can you name the OpenGL object, state, shader stage, or buffer involved in 'Texture Wrapping 4/4'?                                                                                                                                                                                                                                                                                                                                                                                                                                |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 64        | Texel Value Usage                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Source cue     | ■ Older OpenGL versions support several modes for modulating fragment colors by texel colors / (today still relevant to reimplement in shader) - is current fragment with calculated color values / C A T C A / F and alpha values F - is current texture with color values T and alpha values T (or / L C A C                                                                                                                                                                                                                   |
| Commentary     | This slide is about Texel Value Usage. Read it as concrete API state and GPU data movement. Ask which object is bound, which shader stage consumes it, and which state affects the draw call. The visible cue is: ■ Older OpenGL versions support several modes for modulating fragment colors by texel colors / (today still relevant to reimplement in shader) - is current fragment with calculated color values / C A T C A / F and alpha values F - is current texture with color values T and alpha values T (or / L C A C |
| Why it matters | OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Check yourself | Can you name the OpenGL object, state, shader stage, or buffer involved in 'Texel Value Usage'?                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| Page 65        | Texturing in GLSL Shaders 1/3                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| Source cue     | ■ Textures are bound to texture units / ■ Sampler uniforms select the texture unit used by the shader / 1 glBindTextureUnit(0, texture2D); / 2 glBindTextureUnit(1, texture3D); / 4 glUseProgram(p);                                                                                                                                                                                                                                                                                                                             |
| Commentary     | This slide is about Texturing in GLSL Shaders 1/3. Read it as concrete API state and GPU data movement. Ask which object is bound, which shader stage consumes it, and which state affects the draw call. The visible cue is: ■ Textures are bound to texture units / ■ Sampler uniforms select the texture unit used by the shader / 1 glBindTextureUnit(0, texture2D); / 2 glBindTextureUnit(1, texture3D); / 4 glUseProgram(p);                                                                                               |
| Why it matters | OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Check yourself | Can you name the OpenGL object, state, shader stage, or buffer involved in 'Texturing in GLSL Shaders 1/3'?                                                                                                                                                                                                                                                                                                                                                                                                                      |
| Page 66        | Texturing in GLSL Shaders 2/3                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| Source cue     | ■ Vertex Shader / 1 layout(location = 0) in vec3 position; / 2 layout(location = 1) in vec3 texCoord; / 4 out vec3 vTexCoord; / 6 uniform mat4 modelViewProjectionMatrix;                                                                                                                                                                                                                                                                                                                                                        |
| Commentary     | This slide is about Texturing in GLSL Shaders 2/3. Read it as concrete API state and GPU data movement. Ask which object is bound, which shader stage consumes it, and which state affects the draw call. The visible cue is: ■ Vertex Shader / 1 layout(location = 0) in vec3 position; / 2 layout(location = 1) in vec3 texCoord; / 4 out vec3 vTexCoord; / 6 uniform mat4 modelViewProjectionMatrix;                                                                                                                          |
| Why it matters | OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Check yourself | Can you name the OpenGL object, state, shader stage, or buffer involved in 'Texturing in GLSL Shaders 2/3'?                                                                                                                                                                                                                                                                                                                                                                                                                      |

|                |                                                                                                                                                                                                                                                                                                                                                                                                        |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 67        | Texturing in GLSL Shaders 3/3                                                                                                                                                                                                                                                                                                                                                                          |
| Source cue     | ■ Fragment Shader / 1 in vec3 vTexCoord; / 3 out vec4 fragmentColor; / 5 uniform sampler2D texture2D_; / 6 uniform sampler3D texture3D_;                                                                                                                                                                                                                                                               |
| Commentary     | This slide is about Texturing in GLSL Shaders 3/3. Read it as concrete API state and GPU data movement. Ask which object is bound, which shader stage consumes it, and which state affects the draw call. The visible cue is: ■ Fragment Shader / 1 in vec3 vTexCoord; / 3 out vec4 fragmentColor; / 5 uniform sampler2D texture2D_; / 6 uniform sampler3D texture3D_;                                 |
| Why it matters | OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.                                                                                                                                                                                                                                                                                                |
| Check yourself | Can you name the OpenGL object, state, shader stage, or buffer involved in 'Texturing in GLSL Shaders 3/3'?                                                                                                                                                                                                                                                                                            |
| Page 68        | 9.5 Normal Mapping                                                                                                                                                                                                                                                                                                                                                                                     |
| Source cue     | Encoding surface detail in texture space                                                                                                                                                                                                                                                                                                                                                               |
| Commentary     | This slide is about 9.5 Normal Mapping. Read it as local shading. Identify normal, light direction, view direction, material coefficients, and where the computation is evaluated. The visible cue is: Encoding surface detail in texture space                                                                                                                                                        |
| Why it matters | Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.                                                                                                                                                                                                                                                                                                 |
| Check yourself | Can you identify the normal, light vector, view vector, and material term relevant to '9.5 Normal Mapping'?                                                                                                                                                                                                                                                                                            |
| Page 69        | Motivation                                                                                                                                                                                                                                                                                                                                                                                             |
| Source cue     | ■ Render content dichotomy / • Shape: Triangles are used to generate geometric details / k k k / • Appearance: Textures are used to generate material variations ( a , d , s ) / ■ Consequences                                                                                                                                                                                                        |
| Commentary     | This slide is about Motivation. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: ■ Render content dichotomy / • Shape: Triangles are used to generate geometric details / k k k / • Appearance: Textures are used to generate material variations ( a , d , s ) / ■ Consequences |
| Why it matters | Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.                                                                                                                                                                                                                                                                                       |
| Check yourself | Can you explain which samples/fragments are generated by 'Motivation'?                                                                                                                                                                                                                                                                                                                                 |
| Page 70        | Mesh Compression                                                                                                                                                                                                                                                                                                                                                                                       |
| Source cue     | [source]                                                                                                                                                                                                                                                                                                                                                                                               |
| Commentary     | This slide is about Mesh Compression. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: [source]                                                                                                                                                                          |
| Why it matters | Even a sparse slide usually names a relation you must be able to explain in words.                                                                                                                                                                                                                                                                                                                     |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Check yourself | Can you turn 'Mesh Compression' into a causal sentence instead of repeating the slide title?                                                                                                                                                                                                                                                                                                                                                             |
| Page 71        | Normal Mapping 1/2                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Source cue     | ■ Breaks dichotomy by representing geometric details in textures, as texels represent variations in / surface normals / ■ Normal maps accompany diffuse maps with same resolution Material details and geometric / details match                                                                                                                                                                                                                         |
| Commentary     | This slide is about Normal Mapping 1/2. Read it as local shading. Identify normal, light direction, view direction, material coefficients, and where the computation is evaluated. The visible cue is: ■ Breaks dichotomy by representing geometric details in textures, as texels represent variations in / surface normals / ■ Normal maps accompany diffuse maps with same resolution Material details and geometric / details match                  |
| Why it matters | Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.                                                                                                                                                                                                                                                                                                                                                   |
| Check yourself | Can you identify the normal, light vector, view vector, and material term relevant to 'Normal Mapping 1/2'?                                                                                                                                                                                                                                                                                                                                              |
| Page 72        | Normal Mapping 2/2                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Source cue     | $k_n$ / ■ During illumination computation $d$ is fetched from the diffuse map, while $n$ is fetched from the / normal map / $I = k \cdot L \cdot k \cdot L \cdot \max(0, n \cdot l) + k \cdot L \cdot \max(0, v \cdot r) \cdot p / a + d \cdot s$                                                                                                                                                                                                        |
| Commentary     | This slide is about Normal Mapping 2/2. Read it as local shading. Identify normal, light direction, view direction, material coefficients, and where the computation is evaluated. The visible cue is: $k_n$ / ■ During illumination computation $d$ is fetched from the diffuse map, while $n$ is fetched from the / normal map / $I = k \cdot L \cdot k \cdot L \cdot \max(0, n \cdot l) + k \cdot L \cdot \max(0, v \cdot r) \cdot p / a + d \cdot s$ |
| Why it matters | Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.                                                                                                                                                                                                                                                                                                                                                   |
| Check yourself | Can you identify the normal, light vector, view vector, and material term relevant to 'Normal Mapping 2/2'?                                                                                                                                                                                                                                                                                                                                              |
| Page 73        | Normal Maps                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Source cue     | ■ Normal vector components can be positive or negative Components need to be normalized to / [0, 1] [-1, 1] / before storage, and rescaled to after texture fetch / $n' := (n) + 0.5$ / Before storage:                                                                                                                                                                                                                                                  |
| Commentary     | This slide is about Normal Maps. Read it as local shading. Identify normal, light direction, view direction, material coefficients, and where the computation is evaluated. The visible cue is: ■ Normal vector components can be positive or negative Components need to be normalized to / [0, 1] [-1, 1] / before storage, and rescaled to after texture fetch / $n' := (n) + 0.5$ / Before storage:                                                  |
| Why it matters | Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.                                                                                                                                                                                                                                                                                                                                                   |
| Check yourself | Can you identify the normal, light vector, view vector, and material term relevant to 'Normal Maps'?                                                                                                                                                                                                                                                                                                                                                     |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 74        | Tangent Space                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Source cue     | ■ Normal Map vectors align along axis and need to be transformed into view coordinate system / before illumination computation / ■ Tangent coordinate system spanned by / Normal $t / t_s$                                                                                                                                                                                                                                                                                                 |
| Commentary     | This slide is about Tangent Space. Read it as a coordinate-space operation. Name the input space, the matrix or transformation, and the output space before memorizing formulas. The visible cue is: ■ Normal Map vectors align along axis and need to be transformed into view coordinate system / before illumination computation / ■ Tangent coordinate system spanned by / Normal $t / t_s$                                                                                            |
| Why it matters | A wrong coordinate-space assumption can make correct formulas produce wrong images.                                                                                                                                                                                                                                                                                                                                                                                                        |
| Check yourself | Can you state the coordinate space before and after 'Tangent Space'?                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Page 75        | TBN Matrix                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Source cue     | ■ TBN matrix (Tangent, Bitangent, and Normal) is an orthonormal matrix, which transforms tangent / space normal to view space normal / ■ TBN matrix construction (similar to camera transformation) / • TBN matrix contains three perpendicular vectors / • First column vector: tangent                                                                                                                                                                                                   |
| Commentary     | This slide is about TBN Matrix. Read it as a coordinate-space operation. Name the input space, the matrix or transformation, and the output space before memorizing formulas. The visible cue is: ■ TBN matrix (Tangent, Bitangent, and Normal) is an orthonormal matrix, which transforms tangent / space normal to view space normal / ■ TBN matrix construction (similar to camera transformation) / • TBN matrix contains three perpendicular vectors / • First column vector: tangent |
| Why it matters | A wrong coordinate-space assumption can make correct formulas produce wrong images.                                                                                                                                                                                                                                                                                                                                                                                                        |
| Check yourself | Can you state the coordinate space before and after 'TBN Matrix'?                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Page 76        | Untitled slide                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| Source cue     | The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image.                                                                                                                                                                                                                                                                                                                                                                                 |
| Commentary     | This slide is about Untitled slide. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image.                                                                                                                                                              |
| Why it matters | Even a sparse slide usually names a relation you must be able to explain in words.                                                                                                                                                                                                                                                                                                                                                                                                         |
| Check yourself | Can you turn 'Untitled slide' into a causal sentence instead of repeating the slide title?                                                                                                                                                                                                                                                                                                                                                                                                 |
| Page 77        | Tangent Vector Derivation 1/2                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Source cue     | ■ ■ / $t_b$ / Observation: triangle edges can be expressed based on basis given by and / ■ ■ / $e \quad \mathbf{e} = \Delta u \cdot \mathbf{t} + \Delta v \cdot \mathbf{b}$                                                                                                                                                                                                                                                                                                                |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Commentary     | This slide is about Tangent Vector Derivation 1/2. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: $\begin{bmatrix} \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} \end{bmatrix} / t \ b$ / Observation: triangle edges can be expressed based on basis given by and $\begin{bmatrix} \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} \end{bmatrix} / e \begin{bmatrix} \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} \end{bmatrix} = \Delta u \cdot t + \Delta v \cdot b$ |
| Why it matters | Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Check yourself | Can you explain which samples/fragments are generated by 'Tangent Vector Derivation 1/2'?                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |

|                |                                                                                                                                                                                                                                                                                                                               |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 78        | Untitled slide                                                                                                                                                                                                                                                                                                                |
| Source cue     | The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image.                                                                                                                                                                                                                    |
| Commentary     | This slide is about Untitled slide. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image. |
| Why it matters | Even a sparse slide usually names a relation you must be able to explain in words.                                                                                                                                                                                                                                            |
| Check yourself | Can you turn 'Untitled slide' into a causal sentence instead of repeating the slide title?                                                                                                                                                                                                                                    |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 79        | Tangent Vector Derivation 2/2                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| Source cue     | $\begin{bmatrix} \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} \end{bmatrix} / t \ b$ / We can solve this matrix for and $\begin{bmatrix} \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} \end{bmatrix} / t \ t \ t \ \Delta u \ \Delta v \ e \ e \ e / x \ y \ z \ 0 \ 0 \ 0x \ 0y \ 0z$                                                                                                                                                                                                                      |
| Commentary     | This slide is about Tangent Vector Derivation 2/2. Read it as a coordinate-space operation. Name the input space, the matrix or transformation, and the output space before memorizing formulas. The visible cue is: $\begin{bmatrix} \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} \end{bmatrix} / t \ b$ / We can solve this matrix for and $\begin{bmatrix} \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} \end{bmatrix} / t \ t \ t \ \Delta u \ \Delta v \ e \ e \ e / x \ y \ z \ 0 \ 0 \ 0x \ 0y \ 0z$ |
| Why it matters | A wrong coordinate-space assumption can make correct formulas produce wrong images.                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| Check yourself | Can you state the coordinate space before and after 'Tangent Vector Derivation 2/2'?                                                                                                                                                                                                                                                                                                                                                                                                                                           |

|                |                                                                                                                                                                                                                                                                                                                               |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 80        | Untitled slide                                                                                                                                                                                                                                                                                                                |
| Source cue     | The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image.                                                                                                                                                                                                                    |
| Commentary     | This slide is about Untitled slide. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image. |
| Why it matters | Even a sparse slide usually names a relation you must be able to explain in words.                                                                                                                                                                                                                                            |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Check yourself | Can you turn 'Untitled slide' into a causal sentence instead of repeating the slide title?                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Page 81        | Observations                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| Source cue     | $\mathbf{n} \cdot \mathbf{b} = 0$ / $\mathbf{t} \cdot \mathbf{b} = 0$ and are perpendicular to each other as well as to $\mathbf{n}$ only one, or $\mathbf{t}$ , needs to be computed, while the other can be derived through cross product / $\mathbf{n} \times \mathbf{t}$                                                                                                                                                                                                                                                                               |
| Commentary     | <p>This slide is about Observations. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: <math>\mathbf{n} \cdot \mathbf{b} = 0</math> / <math>\mathbf{t} \cdot \mathbf{b} = 0</math> and are perpendicular to each other as well as to <math>\mathbf{n}</math> only one, or <math>\mathbf{t}</math>, needs to be computed, while the other can be derived through cross product / <math>\mathbf{n} \times \mathbf{t}</math></p> |
| Why it matters | Even a sparse slide usually names a relation you must be able to explain in words.                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Check yourself | Can you turn 'Observations' into a causal sentence instead of repeating the slide title?                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| Page 82        | Variations                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Source cue     | <ul style="list-style-type: none"> <li>• Different variations of normal maps exist</li> <li>• Bump maps - specify displacement of surface points in normal direction</li> <li>• Vector offset bump maps - offset vector added to normal</li> <li>• Vector rotation bump maps - normal rotated given angle</li> </ul>                                                                                                                                                                                                                                       |
| Commentary     | <p>This slide is about Variations. Read it as local shading. Identify normal, light direction, view direction, material coefficients, and where the computation is evaluated. The visible cue is: <ul style="list-style-type: none"> <li>• Different variations of normal maps exist</li> <li>• Bump maps - specify displacement of surface points in normal direction</li> <li>• Vector offset bump maps - offset vector added to normal</li> <li>• Vector rotation bump maps - normal rotated given angle</li> </ul></p>                                 |
| Why it matters | Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Check yourself | Can you identify the normal, light vector, view vector, and material term relevant to 'Variations'?                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Page 83        | 9.6 Environment Mapping                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| Source cue     | Sampling directional environment data for reflections and lighting                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Commentary     | <p>This slide is about 9.6 Environment Mapping. Read it as local shading. Identify normal, light direction, view direction, material coefficients, and where the computation is evaluated. The visible cue is: Sampling directional environment data for reflections and lighting</p>                                                                                                                                                                                                                                                                      |
| Why it matters | Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Check yourself | Can you identify the normal, light vector, view vector, and material term relevant to '9.6 Environment Mapping'?                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Page 84        | Environment Mapping Motivation                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Source cue     | <ul style="list-style-type: none"> <li>■ Many reflective and glossy materials show light from their surroundings</li> <li>/ ■ Explicitly tracing all reflection rays is expensive in interactive rendering</li> <li>/ ■ Environment maps provide a compact representation of incident radiance</li> <li>/ ■ Typical use cases</li> <li>• mirror-like reflections</li> </ul>                                                                                                                                                                                                                           |
| Commentary     | <p>This slide is about Environment Mapping Motivation. Read it as an occlusion decision. Decide whether the method reasons about objects, image regions, rays, or per-fragment depth comparisons. The visible cue is:</p> <ul style="list-style-type: none"> <li>■ Many reflective and glossy materials show light from their surroundings</li> <li>/ ■ Explicitly tracing all reflection rays is expensive in interactive rendering</li> <li>/ ■ Environment maps provide a compact representation of incident radiance</li> <li>/ ■ Typical use cases</li> <li>• mirror-like reflections</li> </ul> |
| Why it matters | Visibility decides which generated candidates are actually seen from the current viewpoint.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Check yourself | Can you decide whether 'Environment Mapping Motivation' works per object, per image region, per ray, or per fragment?                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 85        | Environment Mapping Process                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Source cue     | <ul style="list-style-type: none"> <li>■ Environment mapping simulates reflections by sampling an image of the surrounding scene</li> <li>/ ■ The environment is treated as if it were infinitely far away</li> <li>/ ■ The reflected view direction determines which part of the environment is visible</li> <li>/ [Need for Speed no Limits]</li> </ul>                                                                                                                                                                                                                                |
| Commentary     | <p>This slide is about Environment Mapping Process. Read it as sampled data access. Identify coordinates, texture object/state, filtering, mip level, and how the shader interprets the sampled value. The visible cue is:</p> <ul style="list-style-type: none"> <li>■ Environment mapping simulates reflections by sampling an image of the surrounding scene</li> <li>/ ■ The environment is treated as if it were infinitely far away</li> <li>/ ■ The reflected view direction determines which part of the environment is visible</li> <li>/ [Need for Speed no Limits]</li> </ul> |
| Why it matters | Texture sampling is a major source of visual detail and a common source of artifacts.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| Check yourself | Can you identify the sampled data, coordinate, and filtering/state issue in 'Environment Mapping Process'?                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 86        | Environment Mapping Assumptions                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Source cue     | <ul style="list-style-type: none"> <li>■ The environment is assumed to be far away</li> <li>/ • translation of the object does not change the sampled environment</li> <li>/ ■ The reflected object does not reflect itself</li> <li>/ ■ Nearby dynamic objects are usually missing</li> <li>/ ■ The approximation works best for</li> </ul>                                                                                                                                                                                                                                    |
| Commentary     | <p>This slide is about Environment Mapping Assumptions. Read it as sampled data access. Identify coordinates, texture object/state, filtering, mip level, and how the shader interprets the sampled value. The visible cue is:</p> <ul style="list-style-type: none"> <li>■ The environment is assumed to be far away</li> <li>/ • translation of the object does not change the sampled environment</li> <li>/ ■ The reflected object does not reflect itself</li> <li>/ ■ Nearby dynamic objects are usually missing</li> <li>/ ■ The approximation works best for</li> </ul> |
| Why it matters | Texture sampling is a major source of visual detail and a common source of artifacts.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Check yourself | Can you identify the sampled data, coordinate, and filtering/state issue in 'Environment Mapping Assumptions'?                                                                                                                                                                                                                                                                                                                                                                                                 |
| Page 87        | Environment Mapping 2/2                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Source cue     | $V \cdot N$ / ■ At the shaded surface point, the view vector is reflected about the normal / ■ The resulting reflection vector is used to access the environment map / ■ is calculated analog to the Phong illumination model / $R = 2 \cdot (V \cdot N) \cdot N - V$                                                                                                                                                                                                                                          |
| Commentary     | <p>This slide is about Environment Mapping 2/2. Read it as local shading. Identify normal, light direction, view direction, material coefficients, and where the computation is evaluated. The visible cue is: <math>V \cdot N</math> / ■ At the shaded surface point, the view vector is reflected about the normal / ■ The resulting reflection vector is used to access the environment map / ■ is calculated analog to the Phong illumination model / <math>R = 2 \cdot (V \cdot N) \cdot N - V</math></p> |
| Why it matters | Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.                                                                                                                                                                                                                                                                                                                                                                                                         |
| Check yourself | Can you identify the normal, light vector, view vector, and material term relevant to 'Environment Mapping 2/2'?                                                                                                                                                                                                                                                                                                                                                                                               |
| Page 88        | Demo - Cube-map Reflection Vector                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Source cue     | Stage Reflection Vector / object Sphere / showSkybox / reflectionMode Reflection / reflectivity 1                                                                                                                                                                                                                                                                                                                                                                                                              |
| Commentary     | <p>This slide is about Demo - Cube-map Reflection Vector. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: Stage Reflection Vector / object Sphere / showSkybox / reflectionMode Reflection / reflectivity 1</p>                                                                                                                                                                 |
| Why it matters | Even a sparse slide usually names a relation you must be able to explain in words.                                                                                                                                                                                                                                                                                                                                                                                                                             |
| Check yourself | Can you turn 'Demo - Cube-map Reflection Vector' into a causal sentence instead of repeating the slide title?                                                                                                                                                                                                                                                                                                                                                                                                  |
| Page 89        | Environment Mapping Types 1/2                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| Source cue     | ■ Different parameterizations store directional data in different ways / ■ They differ with respect to / • distortion / • filtering behavior / • hardware support                                                                                                                                                                                                                                                                                                                                              |
| Commentary     | <p>This slide is about Environment Mapping Types 1/2. Read it as sampled data access. Identify coordinates, texture object/state, filtering, mip level, and how the shader interprets the sampled value. The visible cue is: ■ Different parameterizations store directional data in different ways / ■ They differ with respect to / • distortion / • filtering behavior / • hardware support</p>                                                                                                             |
| Why it matters | Texture sampling is a major source of visual detail and a common source of artifacts.                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Check yourself | Can you identify the sampled data, coordinate, and filtering/state issue in 'Environment Mapping Types 1/2'?                                                                                                                                                                                                                                                                                                                                                                                                   |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                       |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 90        | Environment Mapping Types 2/2                                                                                                                                                                                                                                                                                                                                                                                         |
| Source cue     | ■ Spherical environment maps / • compact representation in one 2D texture / • strong distortion near the border / • difficult filtering and interpolation behavior / ■ Dual-paraboloid maps                                                                                                                                                                                                                           |
| Commentary     | This slide is about Environment Mapping Types 2/2. Read it as sampled data access. Identify coordinates, texture object/state, filtering, mip level, and how the shader interprets the sampled value. The visible cue is: ■ Spherical environment maps / • compact representation in one 2D texture / • strong distortion near the border / • difficult filtering and interpolation behavior / ■ Dual-paraboloid maps |
| Why it matters | Texture sampling is a major source of visual detail and a common source of artifacts.                                                                                                                                                                                                                                                                                                                                 |
| Check yourself | Can you identify the sampled data, coordinate, and filtering/state issue in 'Environment Mapping Types 2/2'?                                                                                                                                                                                                                                                                                                          |
| Page 91        | Spherical Mapping 1/2                                                                                                                                                                                                                                                                                                                                                                                                 |
| Source cue     | The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image.                                                                                                                                                                                                                                                                                                            |
| Commentary     | This slide is about Spherical Mapping 1/2. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image.                                                                                  |
| Why it matters | Even a sparse slide usually names a relation you must be able to explain in words.                                                                                                                                                                                                                                                                                                                                    |
| Check yourself | Can you turn 'Spherical Mapping 1/2' into a causal sentence instead of repeating the slide title?                                                                                                                                                                                                                                                                                                                     |
| Page 92        | Spherical Mapping 2/2                                                                                                                                                                                                                                                                                                                                                                                                 |
| Source cue     | The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image.                                                                                                                                                                                                                                                                                                            |
| Commentary     | This slide is about Spherical Mapping 2/2. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image.                                                                                  |
| Why it matters | Even a sparse slide usually names a relation you must be able to explain in words.                                                                                                                                                                                                                                                                                                                                    |
| Check yourself | Can you turn 'Spherical Mapping 2/2' into a causal sentence instead of repeating the slide title?                                                                                                                                                                                                                                                                                                                     |
| Page 93        | Spherical Environment Mapping                                                                                                                                                                                                                                                                                                                                                                                         |
| Source cue     | ■ A sphere map stores the environment as seen in a perfectly reflective sphere / ■ It was useful before cube maps became widely supported in graphics hardware / ■ It is mainly relevant today as historical context / ■ Important limitations / • non-uniform distortion                                                                                                                                             |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Commentary     | This slide is about Spherical Environment Mapping. Read it as sampled data access. Identify coordinates, texture object/state, filtering, mip level, and how the shader interprets the sampled value. The visible cue is: ■ A sphere map stores the environment as seen in a perfectly reflective sphere / ■ It was useful before cube maps became widely supported in graphics hardware / ■ It is mainly relevant today as historical context / ■ Important limitations / • non-uniform distortion |
| Why it matters | Texture sampling is a major source of visual detail and a common source of artifacts.                                                                                                                                                                                                                                                                                                                                                                                                               |
| Check yourself | Can you identify the sampled data, coordinate, and filtering/state issue in 'Spherical Environment Mapping'?                                                                                                                                                                                                                                                                                                                                                                                        |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 94        | Sphere Map Distortion                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Source cue     | ■ Sphere-map texels represent differently sized regions of the environment / ■ Distortion is especially problematic in the outer region of the map / ■ Hardware performs linear texture filtering, not spherical interpolation / ■ For modern real-time graphics, this is one reason cube maps are preferred                                                                                                                                                                                                                   |
| Commentary     | This slide is about Sphere Map Distortion. Read it as sampled data access. Identify coordinates, texture object/state, filtering, mip level, and how the shader interprets the sampled value. The visible cue is: ■ Sphere-map texels represent differently sized regions of the environment / ■ Distortion is especially problematic in the outer region of the map / ■ Hardware performs linear texture filtering, not spherical interpolation / ■ For modern real-time graphics, this is one reason cube maps are preferred |
| Why it matters | Texture sampling is a major source of visual detail and a common source of artifacts.                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Check yourself | Can you identify the sampled data, coordinate, and filtering/state issue in 'Sphere Map Distortion'?                                                                                                                                                                                                                                                                                                                                                                                                                           |

|                |                                                                                                                                                                                                                                                                                                                                                  |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 95        | Demo - Environment Map Distortion                                                                                                                                                                                                                                                                                                                |
| Source cue     | Stage Debug Distortion / object Sphere / environmentContent Debug / mapType Spherical map / mappingType Reflection                                                                                                                                                                                                                               |
| Commentary     | This slide is about Demo - Environment Map Distortion. Read it as sampled data access. Identify coordinates, texture object/state, filtering, mip level, and how the shader interprets the sampled value. The visible cue is: Stage Debug Distortion / object Sphere / environmentContent Debug / mapType Spherical map / mappingType Reflection |
| Why it matters | Texture sampling is a major source of visual detail and a common source of artifacts.                                                                                                                                                                                                                                                            |
| Check yourself | Can you identify the sampled data, coordinate, and filtering/state issue in 'Demo - Environment Map Distortion'?                                                                                                                                                                                                                                 |

|            |                                                                                                                                                                                                                                                                                        |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 96    | Sphere Map Interpolation                                                                                                                                                                                                                                                               |
| Source cue | ■ Sphere maps represent non-linear images / ■ Two sphere map texels should be interpolated along an arc / • Graphics hardware does not support spherical interpolation / • Interpolation in the inner area is little distorted / • Interpolation in the outer area is highly distorted |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Commentary     | This slide is about Sphere Map Interpolation. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: ■ Sphere maps represent non-linear images / ■ Two sphere map texels should be interpolated along an arc / • Graphics hardware does not support spherical interpolation / • Interpolation in the inner area is little distorted / • Interpolation in the outer area is highly distorted |
| Why it matters | Even a sparse slide usually names a relation you must be able to explain in words.                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| Check yourself | Can you turn 'Sphere Map Interpolation' into a causal sentence instead of repeating the slide title?                                                                                                                                                                                                                                                                                                                                                                                                                |

|                |                                                                                                                                                                                                                                                                                                                                                                                                        |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 97        | Sphere Map Coordinates                                                                                                                                                                                                                                                                                                                                                                                 |
| Source cue     | ■ Sphere map texture coordinates are calculated in camera coordinates / (0, 0, 0) / • Camera in origin / $V - z + x + y$ / • View direction along the -axis (axis to the right, axis up)                                                                                                                                                                                                               |
| Commentary     | This slide is about Sphere Map Coordinates. Read it as a coordinate-space operation. Name the input space, the matrix or transformation, and the output space before memorizing formulas. The visible cue is: ■ Sphere map texture coordinates are calculated in camera coordinates / (0, 0, 0) / • Camera in origin / $V - z + x + y$ / • View direction along the -axis (axis to the right, axis up) |
| Why it matters | A wrong coordinate-space assumption can make correct formulas produce wrong images.                                                                                                                                                                                                                                                                                                                    |
| Check yourself | Can you state the coordinate space before and after 'Sphere Map Coordinates'?                                                                                                                                                                                                                                                                                                                          |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 98        | Cube Environment Mapping                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Source cue     | ■ Cube maps are represented by 6 square textures forming the sides of a cube / ■ The cube is centered around the shading point or camera / ■ Each face stores the environment as seen through one side of the cube / ■ A 3D direction vector is used to select the face and lookup position                                                                                                                                                                                                                       |
| Commentary     | This slide is about Cube Environment Mapping. Read it as camera geometry. Track how 3D view-space positions become clip coordinates, normalized device coordinates, and finally screen locations. The visible cue is: ■ Cube maps are represented by 6 square textures forming the sides of a cube / ■ The cube is centered around the shading point or camera / ■ Each face stores the environment as seen through one side of the cube / ■ A 3D direction vector is used to select the face and lookup position |
| Why it matters | Projection controls both image composition and depth precision, so it affects visibility and rasterization later.                                                                                                                                                                                                                                                                                                                                                                                                 |
| Check yourself | Can you explain how 'Cube Environment Mapping' changes positions before rasterization?                                                                                                                                                                                                                                                                                                                                                                                                                            |

|            |                                                                                                            |
|------------|------------------------------------------------------------------------------------------------------------|
| Page 99    | Cube Mapping Example 1/2                                                                                   |
| Source cue | The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image. |

|                |                                                                                                                                                                                                                                                                                                                                         |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Commentary     | This slide is about Cube Mapping Example 1/2. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image. |
| Why it matters | Even a sparse slide usually names a relation you must be able to explain in words.                                                                                                                                                                                                                                                      |
| Check yourself | Can you turn 'Cube Mapping Example 1/2' into a causal sentence instead of repeating the slide title?                                                                                                                                                                                                                                    |

|                |                                                                                                                                                                                                                                                                                                                                         |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 100       | Cube Mapping Example 2/2                                                                                                                                                                                                                                                                                                                |
| Source cue     | The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image.                                                                                                                                                                                                                              |
| Commentary     | This slide is about Cube Mapping Example 2/2. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image. |
| Why it matters | Even a sparse slide usually names a relation you must be able to explain in words.                                                                                                                                                                                                                                                      |
| Check yourself | Can you turn 'Cube Mapping Example 2/2' into a causal sentence instead of repeating the slide title?                                                                                                                                                                                                                                    |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                             |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 101       | Cube Mapping Distortion                                                                                                                                                                                                                                                                                                                                                                                                     |
| Source cue     | ■ Cube maps still distort the environment / ■ The distortion is distributed more uniformly than in sphere maps / ■ There are no pole singularities / ■ The main artifacts occur at cube-face boundaries                                                                                                                                                                                                                     |
| Commentary     | This slide is about Cube Mapping Distortion. Read it as sampled data access. Identify coordinates, texture object/state, filtering, mip level, and how the shader interprets the sampled value. The visible cue is: ■ Cube maps still distort the environment / ■ The distortion is distributed more uniformly than in sphere maps / ■ There are no pole singularities / ■ The main artifacts occur at cube-face boundaries |
| Why it matters | Texture sampling is a major source of visual detail and a common source of artifacts.                                                                                                                                                                                                                                                                                                                                       |
| Check yourself | Can you identify the sampled data, coordinate, and filtering/state issue in 'Cube Mapping Distortion'?                                                                                                                                                                                                                                                                                                                      |

|            |                                                                                                                                                                                                                                                                                     |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 102   | Cube Map Texture Coordinates                                                                                                                                                                                                                                                        |
| Source cue | $R = (R_x, R_y, R_z)$ / ■ The reflection vector $x, y, z$ is used as a 3D lookup direction / ■ The cube-map face is selected by the major axis of $R$ / • largest absolute component determines the face / ■ The remaining two components determine the 2D coordinates on that face |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Commentary     | This slide is about Cube Map Texture Coordinates. Read it as a coordinate-space operation. Name the input space, the matrix or transformation, and the output space before memorizing formulas. The visible cue is: $R = (R_x, R_y, R_z)$ / ■ The reflection vector $x y z$ is used as a 3D lookup direction / ■ The cube-map face is selected by the major axis of $R$ / • largest absolute component determines the face / ■ The remaining two components determine the 2D coordinates on that face |
| Why it matters | A wrong coordinate-space assumption can make correct formulas produce wrong images.                                                                                                                                                                                                                                                                                                                                                                                                                   |
| Check yourself | Can you state the coordinate space before and after 'Cube Map Texture Coordinates'?                                                                                                                                                                                                                                                                                                                                                                                                                   |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 103       | Environment Mapping in OpenGL                                                                                                                                                                                                                                                                                                                                                                                                                               |
| Source cue     | ■ In modern OpenGL, reflections are usually stored in a cube map / • six square images form the faces of a virtual cube / • reflection vector is used as a 3D texture coordinate / ■ Typical use cases / • reflective objects                                                                                                                                                                                                                               |
| Commentary     | This slide is about Environment Mapping in OpenGL. Read it as concrete API state and GPU data movement. Ask which object is bound, which shader stage consumes it, and which state affects the draw call. The visible cue is: ■ In modern OpenGL, reflections are usually stored in a cube map / • six square images form the faces of a virtual cube / • reflection vector is used as a 3D texture coordinate / ■ Typical use cases / • reflective objects |
| Why it matters | OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.                                                                                                                                                                                                                                                                                                                                                     |
| Check yourself | Can you name the OpenGL object, state, shader stage, or buffer involved in 'Environment Mapping in OpenGL'?                                                                                                                                                                                                                                                                                                                                                 |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 104       | Cube Map Texture Object                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Source cue     | ■ Cube maps are OpenGL texture objects with target <code>GL_TEXTURE_CUBE_MAP</code> / ■ Each face is addressed by a separate cube-map face target / ■ Use immutable storage for fixed size, format, and mipmap count / 1 <code>GLuint envMap;</code> / 2 <code>glCreateTextures(GL_TEXTURE_CUBE_MAP, 1, &amp;envMap);</code>                                                                                                                                                                                                                         |
| Commentary     | This slide is about Cube Map Texture Object. Read it as concrete API state and GPU data movement. Ask which object is bound, which shader stage consumes it, and which state affects the draw call. The visible cue is: ■ Cube maps are OpenGL texture objects with target <code>GL_TEXTURE_CUBE_MAP</code> / ■ Each face is addressed by a separate cube-map face target / ■ Use immutable storage for fixed size, format, and mipmap count / 1 <code>GLuint envMap;</code> / 2 <code>glCreateTextures(GL_TEXTURE_CUBE_MAP, 1, &amp;envMap);</code> |
| Why it matters | OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Check yourself | Can you name the OpenGL object, state, shader stage, or buffer involved in 'Cube Map Texture Object'?                                                                                                                                                                                                                                                                                                                                                                                                                                                |

|          |                    |
|----------|--------------------|
| Page 105 | Sampling Direction |
|----------|--------------------|

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Source cue     | <p>■ A cube map is sampled with a 3D direction vector / ■ The largest absolute component of the direction selects the cube face / ■ The remaining two components determine the 2D position on that face / ■ For reflection mapping, use the reflected view vector / 1 vec3 V = normalize(cameraPosition - worldPosition);</p>                                                                                                                                                                                                                 |
| Commentary     | <p>This slide is about Sampling Direction. Read it as camera geometry. Track how 3D view-space positions become clip coordinates, normalized device coordinates, and finally screen locations. The visible cue is: ■ A cube map is sampled with a 3D direction vector / ■ The largest absolute component of the direction selects the cube face / ■ The remaining two components determine the 2D position on that face / ■ For reflection mapping, use the reflected view vector / 1 vec3 V = normalize(cameraPosition - worldPosition);</p> |
| Why it matters | <p>Projection controls both image composition and depth precision, so it affects visibility and rasterization later.</p>                                                                                                                                                                                                                                                                                                                                                                                                                      |
| Check yourself | <p>Can you explain how 'Sampling Direction' changes positions before rasterization?</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                       |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 106       | Environment Mapping in GLSL 1/2                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Source cue     | <p>■ Use samplerCube instead of sampler2D / ■ Store positions and normals in world space / ■ Keep the cube map fixed in world space / • otherwise the reflection appears glued to the camera / 1 // Fragment shader</p>                                                                                                                                                                                                                                 |
| Commentary     | <p>This slide is about Environment Mapping in GLSL 1/2. Read it as concrete API state and GPU data movement. Ask which object is bound, which shader stage consumes it, and which state affects the draw call. The visible cue is: ■ Use samplerCube instead of sampler2D / ■ Store positions and normals in world space / ■ Keep the cube map fixed in world space / • otherwise the reflection appears glued to the camera / 1 // Fragment shader</p> |
| Why it matters | <p>OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.</p>                                                                                                                                                                                                                                                                                                                                          |
| Check yourself | <p>Can you name the OpenGL object, state, shader stage, or buffer involved in 'Environment Mapping in GLSL 1/2'?</p>                                                                                                                                                                                                                                                                                                                                    |

|                |                                                                                                                                                                                                                                                                                                                                                                                     |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 107       | Environment Mapping in GLSL 2/2                                                                                                                                                                                                                                                                                                                                                     |
| Source cue     | <p>1 void main() / 2 { / 3 vec3 V = normalize(cameraPosition - vWorldPosition); / 4 vec3 N = normalize(vWorldNormal); / 5 vec3 R = reflect(-V, N);</p>                                                                                                                                                                                                                              |
| Commentary     | <p>This slide is about Environment Mapping in GLSL 2/2. Read it as camera geometry. Track how 3D view-space positions become clip coordinates, normalized device coordinates, and finally screen locations. The visible cue is: 1 void main() / 2 { / 3 vec3 V = normalize(cameraPosition - vWorldPosition); / 4 vec3 N = normalize(vWorldNormal); / 5 vec3 R = reflect(-V, N);</p> |
| Why it matters | <p>Projection controls both image composition and depth precision, so it affects visibility and rasterization later.</p>                                                                                                                                                                                                                                                            |
| Check yourself | <p>Can you explain how 'Environment Mapping in GLSL 2/2' changes positions before rasterization?</p>                                                                                                                                                                                                                                                                                |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 108       | Practical Cube-Map Issues                                                                                                                                                                                                                                                                                                                                                                                                        |
| Source cue     | ■ Cube-map reflections are an approximation / • environment is assumed to be far away / • object does not reflect itself / • other moving objects are usually missing / ■ Important implementation details                                                                                                                                                                                                                       |
| Commentary     | This slide is about Practical Cube-Map Issues. Read it as sampled data access. Identify coordinates, texture object/state, filtering, mip level, and how the shader interprets the sampled value. The visible cue is: ■ Cube-map reflections are an approximation / • environment is assumed to be far away / • object does not reflect itself / • other moving objects are usually missing / ■ Important implementation details |
| Why it matters | Texture sampling is a major source of visual detail and a common source of artifacts.                                                                                                                                                                                                                                                                                                                                            |
| Check yourself | Can you identify the sampled data, coordinate, and filtering/state issue in 'Practical Cube-Map Issues'?                                                                                                                                                                                                                                                                                                                         |
| Page 109       | 1 glEnable(GL_TEXTURE_CUBE_MAP_SEAMLESS);                                                                                                                                                                                                                                                                                                                                                                                        |
| Source cue     | 3 glTextureParameteri(envMap, / Glass Utah teapot rendering; image: Ronak19, CC / BY-SA 4.0. / Extrahierte Tabellen: / [Tabelle 1]                                                                                                                                                                                                                                                                                               |
| Commentary     | This slide is about 1 glEnable(GL_TEXTURE_CUBE_MAP_SEAMLESS);. Read it as sampled data access. Identify coordinates, texture object/state, filtering, mip level, and how the shader interprets the sampled value. The visible cue is: 3 glTextureParameteri(envMap, / Glass Utah teapot rendering; image: Ronak19, CC / BY-SA 4.0. / Extrahierte Tabellen: / [Tabelle 1]                                                         |
| Why it matters | Texture sampling is a major source of visual detail and a common source of artifacts.                                                                                                                                                                                                                                                                                                                                            |
| Check yourself | Can you identify the sampled data, coordinate, and filtering/state issue in '1 glEnable(GL_TEXTURE_CUBE_MAP_SEAMLESS);'?                                                                                                                                                                                                                                                                                                         |
| Page 110       | Cube Mapping Pros and Cons                                                                                                                                                                                                                                                                                                                                                                                                       |
| Source cue     | ■ Advantages / • direct hardware support / • limited distortion / • no pole singularities / • natural representation for directional data                                                                                                                                                                                                                                                                                        |
| Commentary     | This slide is about Cube Mapping Pros and Cons. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: ■ Advantages / • direct hardware support / • limited distortion / • no pole singularities / • natural representation for directional data                                                         |
| Why it matters | Even a sparse slide usually names a relation you must be able to explain in words.                                                                                                                                                                                                                                                                                                                                               |
| Check yourself | Can you turn 'Cube Mapping Pros and Cons' into a causal sentence instead of repeating the slide title?                                                                                                                                                                                                                                                                                                                           |
| Page 111       | Spherical Environment Mapping Assumptions                                                                                                                                                                                                                                                                                                                                                                                        |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Source cue     | <ul style="list-style-type: none"> <li>■ Environment assumed far from object to be textured / ■ Self-reflection not taken into account / • Object to be textured assumed a convex shape(even if it is not geometrically convex) / ■ No other moving objects in the environment / • Interreflection is not taken into account</li> </ul>                                                                                                                                                                                                                                              |
| Commentary     | <p>This slide is about Spherical Environment Mapping Assumptions. Read it as sampled data access. Identify coordinates, texture object/state, filtering, mip level, and how the shader interprets the sampled value. The visible cue is:</p> <ul style="list-style-type: none"> <li>■ Environment assumed far from object to be textured / ■ Self-reflection not taken into account / • Object to be textured assumed a convex shape(even if it is not geometrically convex) / ■ No other moving objects in the environment / • Interreflection is not taken into account</li> </ul> |
| Why it matters | Texture sampling is a major source of visual detail and a common source of artifacts.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Check yourself | Can you identify the sampled data, coordinate, and filtering/state issue in 'Spherical Environment Mapping Assumptions'?                                                                                                                                                                                                                                                                                                                                                                                                                                                             |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 112       | Spherical Environment Mapping Conclusions                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Source cue     | <ul style="list-style-type: none"> <li>■ Advantages / • Contains panoramic image of an environment in a single texture / • Filtering and LoD formation takes place within a single texture / ■ Disadvantages / • Singularity along the outer disc border</li> </ul>                                                                                                                                                                                                                                              |
| Commentary     | <p>This slide is about Spherical Environment Mapping Conclusions. Read it as sampled data access. Identify coordinates, texture object/state, filtering, mip level, and how the shader interprets the sampled value. The visible cue is:</p> <ul style="list-style-type: none"> <li>■ Advantages / • Contains panoramic image of an environment in a single texture / • Filtering and LoD formation takes place within a single texture / ■ Disadvantages / • Singularity along the outer disc border</li> </ul> |
| Why it matters | Texture sampling is a major source of visual detail and a common source of artifacts.                                                                                                                                                                                                                                                                                                                                                                                                                            |
| Check yourself | Can you identify the sampled data, coordinate, and filtering/state issue in 'Spherical Environment Mapping Conclusions'?                                                                                                                                                                                                                                                                                                                                                                                         |

|                |                                                                                                                                                                                                                                                                                                            |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 113       | 9.7 3D Textures and Volume Rendering                                                                                                                                                                                                                                                                       |
| Source cue     | Texturing volumetric data, solid materials, and participating media                                                                                                                                                                                                                                        |
| Commentary     | <p>This slide is about 9.7 3D Textures and Volume Rendering. Read it as local shading. Identify normal, light direction, view direction, material coefficients, and where the computation is evaluated. The visible cue is:</p> <p>Texturing volumetric data, solid materials, and participating media</p> |
| Why it matters | Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.                                                                                                                                                                                                     |
| Check yourself | Can you identify the normal, light vector, view vector, and material term relevant to '9.7 3D Textures and Volume Rendering'?                                                                                                                                                                              |

|            |                                                                                                                                                                                                                         |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 114   | 3D Texturing: Relevant Topics                                                                                                                                                                                           |
| Source cue | <ul style="list-style-type: none"> <li>■ Data model / • Regular voxel grids, channels, formats, memory footprint / ■ Mapping model / (s, t, r) / • Texture coordinates and object-to-texture transformations</li> </ul> |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Commentary     | This slide is about 3D Texturing: Relevant Topics. Read it as a coordinate-space operation. Name the input space, the matrix or transformation, and the output space before memorizing formulas. The visible cue is: ■ Data model / • Regular voxel grids, channels, formats, memory footprint / ■ Mapping model / (s, t, r) / • Texture coordinates and object-to-texture transformations                                                       |
| Why it matters | A wrong coordinate-space assumption can make correct formulas produce wrong images.                                                                                                                                                                                                                                                                                                                                                              |
| Check yourself | Can you state the coordinate space before and after '3D Texturing: Relevant Topics'?                                                                                                                                                                                                                                                                                                                                                             |
| Page 115       | 3D Texture as a Voxel Grid                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Source cue     | ■ A 3D texture stores samples on a regular three-dimensional grid / ■ A sample is often called a voxel / ■ Each voxel can store scalar, vector, or multi-channel data / • Density, temperature, material ID / • Color and opacity                                                                                                                                                                                                                |
| Commentary     | This slide is about 3D Texture as a Voxel Grid. Read it as local shading. Identify normal, light direction, view direction, material coefficients, and where the computation is evaluated. The visible cue is: ■ A 3D texture stores samples on a regular three-dimensional grid / ■ A sample is often called a voxel / ■ Each voxel can store scalar, vector, or multi-channel data / • Density, temperature, material ID / • Color and opacity |
| Why it matters | Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.                                                                                                                                                                                                                                                                                                                                           |
| Check yourself | Can you identify the normal, light vector, view vector, and material term relevant to '3D Texture as a Voxel Grid'?                                                                                                                                                                                                                                                                                                                              |
| Page 116       | Object Space to Texture Space                                                                                                                                                                                                                                                                                                                                                                                                                    |
| Source cue     | ■ 3D texture coordinates are usually derived from object-space position / ■ For a volume-aligned bounding box / $p - p / \text{object min} / (s, t, r) =$                                                                                                                                                                                                                                                                                        |
| Commentary     | This slide is about Object Space to Texture Space. Read it as a coordinate-space operation. Name the input space, the matrix or transformation, and the output space before memorizing formulas. The visible cue is: ■ 3D texture coordinates are usually derived from object-space position / ■ For a volume-aligned bounding box / $p - p / \text{object min} / (s, t, r) =$                                                                   |
| Why it matters | A wrong coordinate-space assumption can make correct formulas produce wrong images.                                                                                                                                                                                                                                                                                                                                                              |
| Check yourself | Can you state the coordinate space before and after 'Object Space to Texture Space'?                                                                                                                                                                                                                                                                                                                                                             |
| Page 117       | 3D Texture Access                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Source cue     | ■ For each fragment, texture coordinates are interpolated / ■ A 3D lookup fetches the neighboring voxel values / ■ Linear filtering in 3D performs trilinear interpolation                                                                                                                                                                                                                                                                       |

|                |                                                                                                                                                                                                                                                                                                                                                                                     |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Commentary     | This slide is about 3D Texture Access. Read it as a coordinate-space operation. Name the input space, the matrix or transformation, and the output space before memorizing formulas. The visible cue is: ■ For each fragment, texture coordinates are interpolated / ■ A 3D lookup fetches the neighboring voxel values / ■ Linear filtering in 3D performs trilinear interpolation |
| Why it matters | A wrong coordinate-space assumption can make correct formulas produce wrong images.                                                                                                                                                                                                                                                                                                 |
| Check yourself | Can you state the coordinate space before and after '3D Texture Access'?                                                                                                                                                                                                                                                                                                            |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 118       | Trilinear Interpolation                                                                                                                                                                                                                                                                                                                                                                                                                      |
| Source cue     | ■ Trilinear interpolation is linear interpolation along three axes / ■ It blends the eight voxel values surrounding the lookup position / ■ Conceptually / • interpolate along / • interpolate the results along                                                                                                                                                                                                                             |
| Commentary     | This slide is about Trilinear Interpolation. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: ■ Trilinear interpolation is linear interpolation along three axes / ■ It blends the eight voxel values surrounding the lookup position / ■ Conceptually / • interpolate along / • interpolate the results along |
| Why it matters | Even a sparse slide usually names a relation you must be able to explain in words.                                                                                                                                                                                                                                                                                                                                                           |
| Check yourself | Can you turn 'Trilinear Interpolation' into a causal sentence instead of repeating the slide title?                                                                                                                                                                                                                                                                                                                                          |

|                |                                                                                                                                                                                                                                                                                                                                                                                   |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 119       | $f = (1 - \alpha) \cdot f + \alpha \cdot f$                                                                                                                                                                                                                                                                                                                                       |
| Source cue     | 00 000 100 / $f = (1 - \alpha) \cdot f + \alpha \cdot f$ / 01 001 101 / $f = (1 - \alpha) \cdot f + \alpha \cdot f$ / 10 010 110                                                                                                                                                                                                                                                  |
| Commentary     | This slide is about $f = (1 - \alpha) \cdot f + \alpha \cdot f$ . Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: 00 000 100 / $f = (1 - \alpha) \cdot f + \alpha \cdot f$ / 01 001 101 / $f = (1 - \alpha) \cdot f + \alpha \cdot f$ / 10 010 110 |
| Why it matters | Even a sparse slide usually names a relation you must be able to explain in words.                                                                                                                                                                                                                                                                                                |
| Check yourself | Can you turn ' $f = (1 - \alpha) \cdot f + \alpha \cdot f$ ' into a causal sentence instead of repeating the slide title?                                                                                                                                                                                                                                                         |

|            |                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 120   | 3D Texture Memory Footprint                                                                                                                                                                                                                                                                                                                                                                                                          |
| Source cue | ■ Memory grows cubically with resolution / memory = $w \cdot h \cdot d \cdot \text{bytes per voxel}$ / Resolution R8 R16F RGBA8 RGBA16F / 1 B/voxel 2 B/voxel 4 B/voxel 8 B/voxel / 2 MiB 4 MiB 8 MiB 16 MiB                                                                                                                                                                                                                         |
| Commentary | This slide is about 3D Texture Memory Footprint. Read it as sampled data access. Identify coordinates, texture object/state, filtering, mip level, and how the shader interprets the sampled value. The visible cue is: ■ Memory grows cubically with resolution / memory = $w \cdot h \cdot d \cdot \text{bytes per voxel}$ / Resolution R8 R16F RGBA8 RGBA16F / 1 B/voxel 2 B/voxel 4 B/voxel 8 B/voxel / 2 MiB 4 MiB 8 MiB 16 MiB |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Why it matters | Texture sampling is a major source of visual detail and a common source of artifacts.                                                                                                                                                                                                                                                                                                                                                                        |
| Check yourself | Can you identify the sampled data, coordinate, and filtering/state issue in '3D Texture Memory Footprint'?                                                                                                                                                                                                                                                                                                                                                   |
| Page 121       | 3D Mipmaps and Filtering                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Source cue     | ■ 3D textures can use mipmaps analogously to 2D textures / ■ Each mip level halves width, height, and depth / $k + 1$ k / ■ Level contains one eighth of the voxels of level / ■ Useful for                                                                                                                                                                                                                                                                  |
| Commentary     | This slide is about 3D Mipmaps and Filtering. Read it as an occlusion decision. Decide whether the method reasons about objects, image regions, rays, or per-fragment depth comparisons. The visible cue is: ■ 3D textures can use mipmaps analogously to 2D textures / ■ Each mip level halves width, height, and depth / $k + 1$ k / ■ Level contains one eighth of the voxels of level / ■ Useful for                                                     |
| Why it matters | Visibility decides which generated candidates are actually seen from the current viewpoint.                                                                                                                                                                                                                                                                                                                                                                  |
| Check yourself | Can you decide whether '3D Mipmaps and Filtering' works per object, per image region, per ray, or per fragment?                                                                                                                                                                                                                                                                                                                                              |
| Page 122       | Solid Texturing                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Source cue     | ■ 3D textures can define material throughout the interior of an object / ■ Surface fragments sample the material at their 3D position / ■ This avoids seams caused by 2D UV unwrapping / ■ Useful for materials with volumetric structure / • wood grain                                                                                                                                                                                                     |
| Commentary     | This slide is about Solid Texturing. Read it as local shading. Identify normal, light direction, view direction, material coefficients, and where the computation is evaluated. The visible cue is: ■ 3D textures can define material throughout the interior of an object / ■ Surface fragments sample the material at their 3D position / ■ This avoids seams caused by 2D UV unwrapping / ■ Useful for materials with volumetric structure / • wood grain |
| Why it matters | Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.                                                                                                                                                                                                                                                                                                                                                       |
| Check yourself | Can you identify the normal, light vector, view vector, and material term relevant to 'Solid Texturing'?                                                                                                                                                                                                                                                                                                                                                     |
| Page 123       | Volumetric Data as Texture Data                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Source cue     | ■ Volume data is often represented as a scalar field / $\rho = f(x, y, z)$ / ■ Examples / • CT or MRI intensity / • smoke density                                                                                                                                                                                                                                                                                                                            |
| Commentary     | This slide is about Volumetric Data as Texture Data. Read it as sampled data access. Identify coordinates, texture object/state, filtering, mip level, and how the shader interprets the sampled value. The visible cue is: ■ Volume data is often represented as a scalar field / $\rho = f(x, y, z)$ / ■ Examples / • CT or MRI intensity / • smoke density                                                                                                |
| Why it matters | Texture sampling is a major source of visual detail and a common source of artifacts.                                                                                                                                                                                                                                                                                                                                                                        |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Check yourself | Can you identify the sampled data, coordinate, and filtering/state issue in 'Volumetric Data as Texture Data'?                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| Page 124       | Transfer Functions                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| Source cue     | <ul style="list-style-type: none"> <li>■ A transfer function maps data values to optical properties / <math>T(p) = (c, \alpha)</math></li> <li>■ Color emphasizes material classes or value ranges</li> <li>■ Opacity controls which structures become visible</li> <li>■ Common choices</li> </ul>                                                                                                                                                                                                                                                                                                                 |
| Commentary     | This slide is about Transfer Functions. Read it as local shading. Identify normal, light direction, view direction, material coefficients, and where the computation is evaluated. The visible cue is: <ul style="list-style-type: none"> <li>■ A transfer function maps data values to optical properties / <math>T(p) = (c, \alpha)</math></li> <li>■ Color emphasizes material classes or value ranges</li> <li>■ Opacity controls which structures become visible</li> <li>■ Common choices</li> </ul>                                                                                                          |
| Why it matters | Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Check yourself | Can you identify the normal, light vector, view vector, and material term relevant to 'Transfer Functions'?                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Page 125       | Volume Rendering with 3D Textures                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| Source cue     | <ul style="list-style-type: none"> <li>■ 3D grid data is stored in a 3D texture</li> <li>■ Slice-based volume rendering draws proxy geometry through the volume</li> <li>■ Slices are usually orthogonal to the view direction</li> <li>■ Each slice samples the 3D texture and applies the transfer function</li> <li>■ Slices are blended back to front or front to back</li> </ul>                                                                                                                                                                                                                               |
| Commentary     | This slide is about Volume Rendering with 3D Textures. Read it as sampled data access. Identify coordinates, texture object/state, filtering, mip level, and how the shader interprets the sampled value. The visible cue is: <ul style="list-style-type: none"> <li>■ 3D grid data is stored in a 3D texture</li> <li>■ Slice-based volume rendering draws proxy geometry through the volume</li> <li>■ Slices are usually orthogonal to the view direction</li> <li>■ Each slice samples the 3D texture and applies the transfer function</li> <li>■ Slices are blended back to front or front to back</li> </ul> |
| Why it matters | Texture sampling is a major source of visual detail and a common source of artifacts.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| Check yourself | Can you identify the sampled data, coordinate, and filtering/state issue in 'Volume Rendering with 3D Textures'?                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| Page 126       | Front-to-Back Alpha Compositing                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Source cue     | <ul style="list-style-type: none"> <li>■ Volume rendering approximates absorption and emission along a ray / <math>c \alpha</math></li> <li>■ For a sample with color <math>i</math> and opacity <math>i</math> / <math>C = C + (1 - \alpha) \cdot \alpha \cdot c</math> / out in <math>i</math></li> </ul>                                                                                                                                                                                                                                                                                                         |
| Commentary     | This slide is about Front-to-Back Alpha Compositing. Read it as an occlusion decision. Decide whether the method reasons about objects, image regions, rays, or per-fragment depth comparisons. The visible cue is: <ul style="list-style-type: none"> <li>■ Volume rendering approximates absorption and emission along a ray / <math>c \alpha</math></li> <li>■ For a sample with color <math>i</math> and opacity <math>i</math> / <math>C = C + (1 - \alpha) \cdot \alpha \cdot c</math> / out in <math>i</math></li> </ul>                                                                                     |
| Why it matters | Visibility decides which generated candidates are actually seen from the current viewpoint.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Check yourself | Can you decide whether 'Front-to-Back Alpha Compositing' works per object, per image region, per ray, or per fragment?                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Page 127       | Ray Marching Volume Rendering                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Source cue     | <ul style="list-style-type: none"> <li>■ Instead of drawing slices, cast one ray per fragment through the volume</li> <li>■ Repeatedly sample the 3D texture along the ray</li> <li>■ Apply the transfer function at every sample</li> <li>■ Composite samples until the ray exits the volume or becomes opaque</li> </ul> <pre>1 for (float t = tEntry; t &lt; tExit; t += stepSize) {</pre>                                                                                                                                                                                                                          |
| Commentary     | <p>This slide is about Ray Marching Volume Rendering. Read it as an occlusion decision. Decide whether the method reasons about objects, image regions, rays, or per-fragment depth comparisons. The visible cue is:</p> <ul style="list-style-type: none"> <li>■ Instead of drawing slices, cast one ray per fragment through the volume</li> <li>■ Repeatedly sample the 3D texture along the ray</li> <li>■ Apply the transfer function at every sample</li> <li>■ Composite samples until the ray exits the volume or becomes opaque</li> </ul> <pre>1 for (float t = tEntry; t &lt; tExit; t += stepSize) {</pre> |
| Why it matters | Visibility decides which generated candidates are actually seen from the current viewpoint.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| Check yourself | Can you decide whether 'Ray Marching Volume Rendering' works per object, per image region, per ray, or per fragment?                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| Page 128       | Gradients and Volume Lighting                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Source cue     | <ul style="list-style-type: none"> <li>■ Gradients estimate local orientation in the scalar field</li> <li>■ Central differences approximate the gradient</li> </ul> $\rho(x + \Delta, y, z) - \rho(x - \Delta, y, z) / \nabla \rho \approx \rho(x, y + \Delta, z) - \rho(x, y - \Delta, z)$                                                                                                                                                                                                                                                                                                                           |
| Commentary     | <p>This slide is about Gradients and Volume Lighting. Read it as local shading. Identify normal, light direction, view direction, material coefficients, and where the computation is evaluated. The visible cue is:</p> <ul style="list-style-type: none"> <li>■ Gradients estimate local orientation in the scalar field</li> <li>■ Central differences approximate the gradient</li> </ul> $\rho(x + \Delta, y, z) - \rho(x - \Delta, y, z) / \nabla \rho \approx \rho(x, y + \Delta, z) - \rho(x, y - \Delta, z)$                                                                                                  |
| Why it matters | Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Check yourself | Can you identify the normal, light vector, view vector, and material term relevant to 'Gradients and Volume Lighting'?                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Page 129       | Quality and Performance Trade-Offs                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Source cue     | <ul style="list-style-type: none"> <li>■ Step size</li> <li>• smaller steps improve quality but increase cost</li> <li>• larger steps are faster but can miss thin structures</li> <li>■ Opacity correction</li> <li>• keeps appearance stable when step size changes</li> </ul>                                                                                                                                                                                                                                                                                                                                       |
| Commentary     | <p>This slide is about Quality and Performance Trade-Offs. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is:</p> <ul style="list-style-type: none"> <li>■ Step size</li> <li>• smaller steps improve quality but increase cost</li> <li>• larger steps are faster but can miss thin structures</li> <li>■ Opacity correction</li> <li>• keeps appearance stable when step size changes</li> </ul>                                                                                         |
| Why it matters | Even a sparse slide usually names a relation you must be able to explain in words.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Check yourself | Can you turn 'Quality and Performance Trade-Offs' into a causal sentence instead of repeating the slide title?                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Page 130       | 3D Texturing Takeaways                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| Source cue     | <ul style="list-style-type: none"> <li>■ 3D textures generalize 2D texture lookup from surfaces to volumes / ■ They are useful for solid materials, volumetric effects, and sampled scientific data / ■ Trilinear interpolation provides smooth access to voxel grids / ■ Volume rendering turns sampled data into color and opacity along viewing rays / ■ Transfer functions and compositing determine what becomes visible</li> </ul>                                                                                                                                                                 |
| Commentary     | <p>This slide is about 3D Texturing Takeaways. Read it as an occlusion decision. Decide whether the method reasons about objects, image regions, rays, or per-fragment depth comparisons. The visible cue is: ■ 3D textures generalize 2D texture lookup from surfaces to volumes / ■ They are useful for solid materials, volumetric effects, and sampled scientific data / ■ Trilinear interpolation provides smooth access to voxel grids / ■ Volume rendering turns sampled data into color and opacity along viewing rays / ■ Transfer functions and compositing determine what becomes visible</p> |
| Why it matters | Visibility decides which generated candidates are actually seen from the current viewpoint.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Check yourself | Can you decide whether '3D Texturing Takeaways' works per object, per image region, per ray, or per fragment?                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| Page 131       | ■ Textures are sampled GPU data sources used by shaders                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| Source cue     | <ul style="list-style-type: none"> <li>• Color, opacity, normals, roughness, height, masks, and volume density / ■ Texture coordinates define where texture data is sampled / (s, t, r) / • UV mapping, texture transformations, and 3D coordinates / ■ Texture formats, channel semantics, and color space determine correct interpretation</li> </ul>                                                                                                                                                                                                                                                  |
| Commentary     | <p>This slide is about ■ Textures are sampled GPU data sources used by shaders. Read it as concrete API state and GPU data movement. Ask which object is bound, which shader stage consumes it, and which state affects the draw call. The visible cue is: • Color, opacity, normals, roughness, height, masks, and volume density / ■ Texture coordinates define where texture data is sampled / (s, t, r) / • UV mapping, texture transformations, and 3D coordinates / ■ Texture formats, channel semantics, and color space determine correct interpretation</p>                                     |
| Why it matters | OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| Check yourself | Can you name the OpenGL object, state, shader stage, or buffer involved in '■ Textures are sampled GPU data sources used by shaders'?                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| Page 132       | Literature and other sources used in this chapter                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Source cue     | The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |

|                |                                                                                                                                                                                                                                                                                                                                                                  |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Commentary     | This slide is about Literature and other sources used in this chapter. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image. |
| Why it matters | Even a sparse slide usually names a relation you must be able to explain in words.                                                                                                                                                                                                                                                                               |
| Check yourself | Can you turn 'Literature and other sources used in this chapter' into a causal sentence instead of repeating the slide title?                                                                                                                                                                                                                                    |

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page 133       | ■ Text Books                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Source cue     | • J. Foley, A. van Dam, S. Fine"Computer Graphics: Principles and Practice (3About Edition)", / Addison-Wesley 2013. / • T. Akenine-Möller, E. Haines: Real-Time Rendering, Addison-Wesley. / • P. Shirley, M. AshikhminS. Marschner: Fundamentals of Computer Graphics (3rd ed. Edition), AK / ■ Others                                                                                                                                                                                                                  |
| Commentary     | This slide is about ■ Text Books. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: • J. Foley, A. van Dam, S. Fine"Computer Graphics: Principles and Practice (3About Edition)", / Addison-Wesley 2013. / • T. Akenine-Möller, E. Haines: Real-Time Rendering, Addison-Wesley. / • P. Shirley, M. AshikhminS. Marschner: Fundamentals of Computer Graphics (3rd ed. Edition), AK / ■ Others |
| Why it matters | Even a sparse slide usually names a relation you must be able to explain in words.                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Check yourself | Can you turn '■ Text Books' into a causal sentence instead of repeating the slide title?                                                                                                                                                                                                                                                                                                                                                                                                                                  |

## 09.1 Texture Objects, Texels, Formats, and Color Space

Source location: Section 9.1

### Commentary

A texture object stores sampled data on the GPU. A texel is one stored texture sample. The texture format decides what channels and numeric representation the samples use. Color space matters because color values may be stored nonlinearly, especially for sRGB images.

The key shift is to stop thinking of texture as only a picture. Textures can store color, normals, depth, lookup data, environment information, or volume data. The shader decides how the sampled values are interpreted.

### Mental model

A texture is a GPU-accessible sampled data field.

### Check yourself

Why can the same texture mechanism store both color maps and normal maps?

## 09.2 Texture Coordinates, UV Mapping, and Wrapping

Source location: Section 9.2

## Commentary

Texture coordinates map surface points to locations in texture space. UV coordinates are usually interpolated across primitives during rasterization and then used by the fragment shader to sample a texture.

Wrapping rules decide what happens outside the normal coordinate range. Repeat, clamp, mirrored repeat, and border behavior are not visual afterthoughts; they define the sampling function outside the base domain.

## Mental model

UVs are the address system that lets a fragment look up texture data.

## Check yourself

What artifact might appear if a wrapping mode is wrong at the edge of a surface?

# 09.3 Sampling, Filtering, Mipmaps, and Anisotropy

Source location: Section 9.3

## Commentary

Sampling turns continuous texture coordinates into values from discrete texels. Nearest filtering selects a nearby texel and can look blocky. Linear filtering blends nearby texels and looks smoother. Minification is harder because many texels may map to one pixel.

Mipmaps store prefiltered lower-resolution versions of a texture. They reduce aliasing and shimmer when textures are seen at small scale. Anisotropic filtering improves quality when a texture is viewed at a steep angle where footprint shape is elongated.

## Mental model

Filtering reconstructs values; mipmaps choose an appropriate scale before reconstruction.

## Check yourself

Why does a distant checkerboard shimmer without mipmapping?

# 09.4 Modern OpenGL Texture Pipeline

Source location: Section 9.4

## Commentary

In modern OpenGL, textures are represented by texture objects, bound to texture units, connected to sampler uniforms, and accessed in shaders. The shader does not sample a filename; it samples a bound texture through a sampler.

Texture bugs often come from state mismatches: wrong active texture unit, wrong sampler uniform, missing mipmaps for a mipmap filter, wrong wrap mode, or wrong internal format.

## Mental model

OpenGL texturing is a chain: object data -> texture unit -> sampler uniform -> shader lookup.

## Check yourself

Why can a texture appear black when the shader code is mathematically correct?

## 09.5 Normal Mapping

Source location: Section 9.5

### Commentary

Normal mapping stores normal directions in a texture so that lighting can vary at a finer scale than the geometry. The surface may have few triangles, but the shader receives detailed normals per fragment.

The hard part is coordinate space. Normal maps are often defined in tangent space, so the shader must transform or interpret them with the correct tangent, bitangent, and normal basis.

### Mental model

Normal mapping changes the lighting normal, not the actual mesh silhouette.

## Check yourself

Why does normal mapping not change the geometric outline of an object?

## 09.6 Environment Mapping and 3D Textures

Source location: Sections 9.6-9.7

### Commentary

Environment mapping uses textures to represent surrounding illumination or reflections. A direction, not a surface UV alone, can be used to sample an environment map. This is a texture lookup driven by view/reflection geometry.

3D textures and volume rendering extend the same sampled-data idea into three dimensions. Instead of sampling a 2D image, the shader samples a volume. This is useful for data such as medical scans, density fields, or procedural volumetric effects.

### Mental model

Textures are general sampled data; the coordinate dimensionality depends on the problem.

## Check yourself

What coordinate type would you use to sample a 3D texture?

## End-of-lecture summary

If you remember only one thing from Lecture 09, remember this: This lecture treats textures as sampled data for shading, not merely images glued onto objects. It connects UV coordinates, sampling, filtering, mipmaps, OpenGL texture state, and advanced texture uses.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.